

웹 서버의 엔진과 설계도

Servlet과 JSP의 내부 동작 원리와 파이프라인

Engine
(Servlet)

Blueprint
(JSP)

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11 public class MyServlet extends HttpServlet {  
12  
13     @Override  
14     public void naain(request) {  
15  
16         request.getParameter("data");  
17     }  
18     ...  
19 }  
20
```

```
<%@ page language="java"  
contentType="text/html; charset=UTF-8"  
%>  
  
<h1>Welcome, <%= username %></h1>
```

```
<h1>Welcome, <%= username %></h1>
```

```
public class MyServlet extends {  
    ...  
    request.getParameter("data");  
}
```

서블릿(Servlet)과 JSP: 자바 웹 개발의 핵심 이해

서블릿(Servlet)은 자바를 이용해 웹 요청을 처리하는 기술이며, JSP는 이를 HTML 중심으로 발전시킨 기술입니다. 모든 JSP는 실행 시 서블릿으로 변환되며, 이들은 공통된 생명주기를 통해 웹 애플리케이션의 핵심 로직과 화면을 담당합니다.

서블릿과 JSP의 생명주기 (Lifecycle)

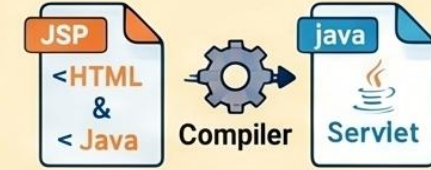


초기화 및 생성 (init)

최초 요청 시 객체가 생성되며, DB 연결 등 초기 설정 작업을 단 한 번 수행합니다.



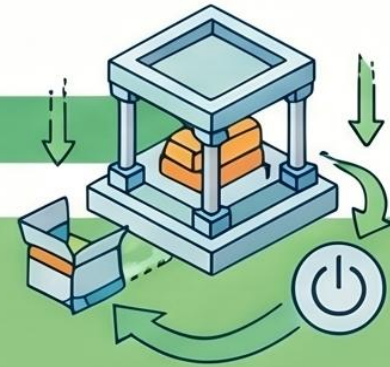
JSP도 결국 서블릿이다



JSP 자원은 실행 시 전테이너에 비해 .java 서블릿 코드로 변환 및 컴파일되어 실행됩니다.

요청 처리 및 실행 (service)

클라이언트 요청마다 실행되며, GET/POST 방식에 맞는 비즈니스 로직을 수행합니다.



소멸 및 자원 해제 (destroy)

서버 종료 시 호출되어 사용 중이던 DB 연결이나 자원을 안전하게 반납합니다.

Servlet (Controller: 비즈니스 로직)

```
1 public java {  
2   public honiol {  
3     public serahz {  
4       chie.sozamO }  
5       unte.girawiohzO;  
6     }  
7   }  
8   public void {  
9     ceetm proussioO;  
10    return leauitO;  
11  }  
12 }
```

JSP (View: 화면 구성)



역할의 분리 (MVC 패턴의 기초)

데이터 저장 범위 (Scope)

page

request

session

application

데이터는 page < request < session < application 순으로 유지 범위가 넓어집니다.

Servlet vs JSP 핵심 차이점

구분	Servlet	JSP
중심 언어	Java 중심	HTML 중심
주요 역할	Controller (비즈니스 로직)	View (화면 구성)
화면 작성	자바 코드 내 HTML 문자열 출력	HTML 내 자바 코드 삽입

한눈에 보는 JSP 웹 개발 핵심 가이드

JSP 웹 애플리케이션 개발에서 사용자 상태를 유지하는 Cookie와 Session의 차이점을 파악하고, 시스템 성능을 높이는 Connection Pool 기술과 유지보수에 최적화된 MVC 패턴의 구조를 요약하여 설명합니다.

상대 유지 및 효율적인 데이터 관리

Cookie vs Session: 저장 위치가 핵심

쿠키는 브라우저에, 세션은 서버에 정보를 저장하며 보안과 성능의 균형을 맞춥니다.

Cookie (쿠키)



- 저장 위치: 브라우저 (클라이언트)
- 보안상: 낮음 (탈취/변조 가능)
- 사용 사례: 어디저장, 최근 본 상품



브라우저

- 저장 위치: 브라우저 (클라이언트)
- 보안상: 낮음 (탈취/변조 가능)
- 사용 사례: 어디저장, 최근 본 상품

Session (세션)



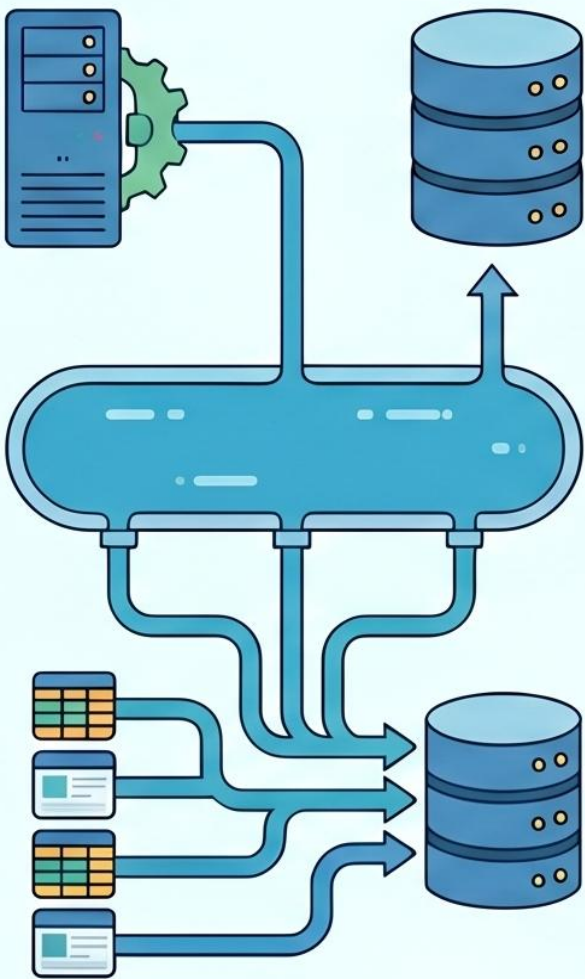
- 저장 위치: 웹 서버
- 보안상: 높음 (상태적으로 안전)
- 사용 사례: 로그인 인증, 장바구니



웹 서버

- 저장 위치: 웹 서버
- 보안상: 높음 (상태적으로 안전)
- 사용 사례: 로그인 인증, 장바구니

Connection Pool로 DB 성능 극대화

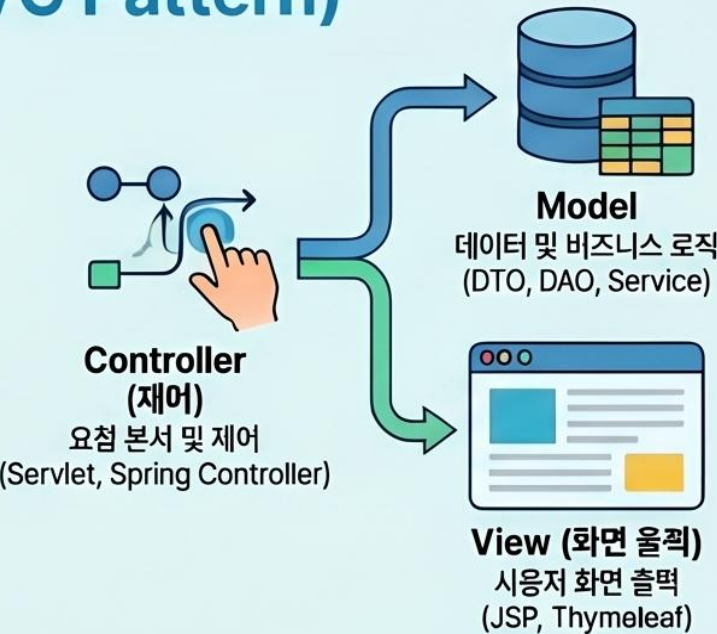


매번 연결을 생성하지 않고 미리 만든 연결을 재사용하여 서버 부하를 획기적으로 줄입니다.

현대적 설계 구조(MVC Pattern)

Model-View-Controller의 역할 분리

데이터 처리(Model), 화면 출력(View), 요청 제어(Controller)를 나누어 유지보수성을 높입니다.



Servlet과 JSP의 분업화

JSP는 화면만 담당하는 뷰로 사용합니다.



Servlet
(컨트롤러)
로직을 담당

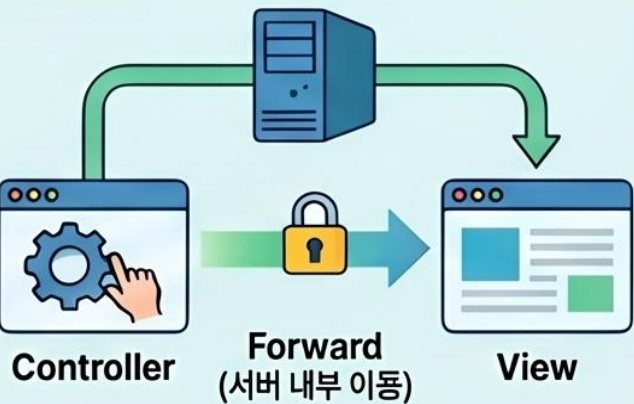


JSP
(뷰)
화면만 담당

Servlet은 로직을 담당하는 컨트롤러로, JSP는 화면만 담당하는 뷰로 사용됩니다.

Forward 방식을 통한 데이터 유지

컨트롤러에서 처리한 데이터를 뷰까지 유지



서버 내부 이동인 Forward를 사용하여 컨트롤러에서 처리한 데이터를 뷰까지 안전하게 전달합니다.

한눈에 이해하는 JSP(JavaServer Pages): 동적 웹 페이지의 핵심

JSP의 정의와 동작 원리



HTML과 Java의 결합, JSP

HTML 내부에 자바 코드를 실행하여 서버에서 실시간으로
동적 콘텐츠를 생성합니다.

JSP(서버) vs JavaScript(클라이언트)



JSP(서버)

JSP는 서버에서
DB 연동을 담당합니다.



JavaScript(클라이언트)

JS는 브라우저에서
UI 이벤트 처리를 담당합니다.

효율적인 개발을 위한 구성 요소



로직 구현



출력



변수 선언

3가지 핵심 스크립트 요소

스크립트릿, 표한식, 선언을 통해
로직을 구현합니다.



편리한 내장 객체와 EL/JSTL

별도 선언 없이 request, session 등 객체를
사용하며, 태그 기반 언어로 가독성을 높입니다.



EL/JSTL

JSP와 JavaScript의 근본적인 차이점

구분	JSP	JavaScript
실행 위치	서버 (Server) 측	클라이언트 (Browser) 측
주요 역할	DB 연동, 비즈니스 로직 처리	화면 이벤트, 유효성 검사 등 UI 제어
페이지 변화	서버 통신 시 페이지 새로고침 발생	새로고침 없이 동적 변화 가능



Model



View

MVC 패턴의 핵심 'View'

비즈니스 로직과 화면을 분리하여 코드의
가독성과 유지보수성을 극대화합니다.

JSP의 6단계 실행 프로세스



클라이언트의 요청이 컨테이너에 도달하는 과정

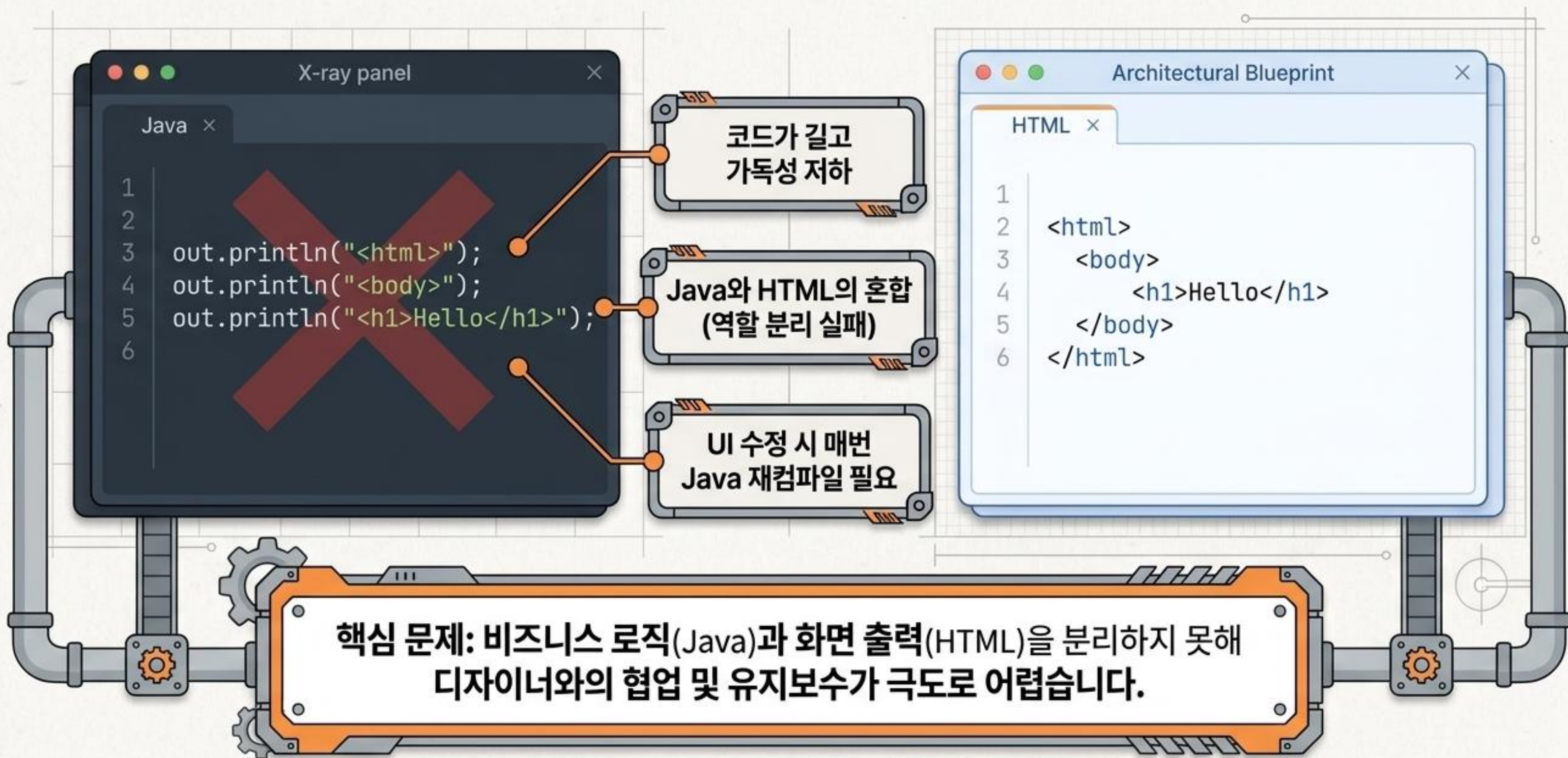


웹 서버(또는 톰캣 자체)가 URL을 확인하고 해당 요청을 처리할 **서블릿**을 매핑합니다.

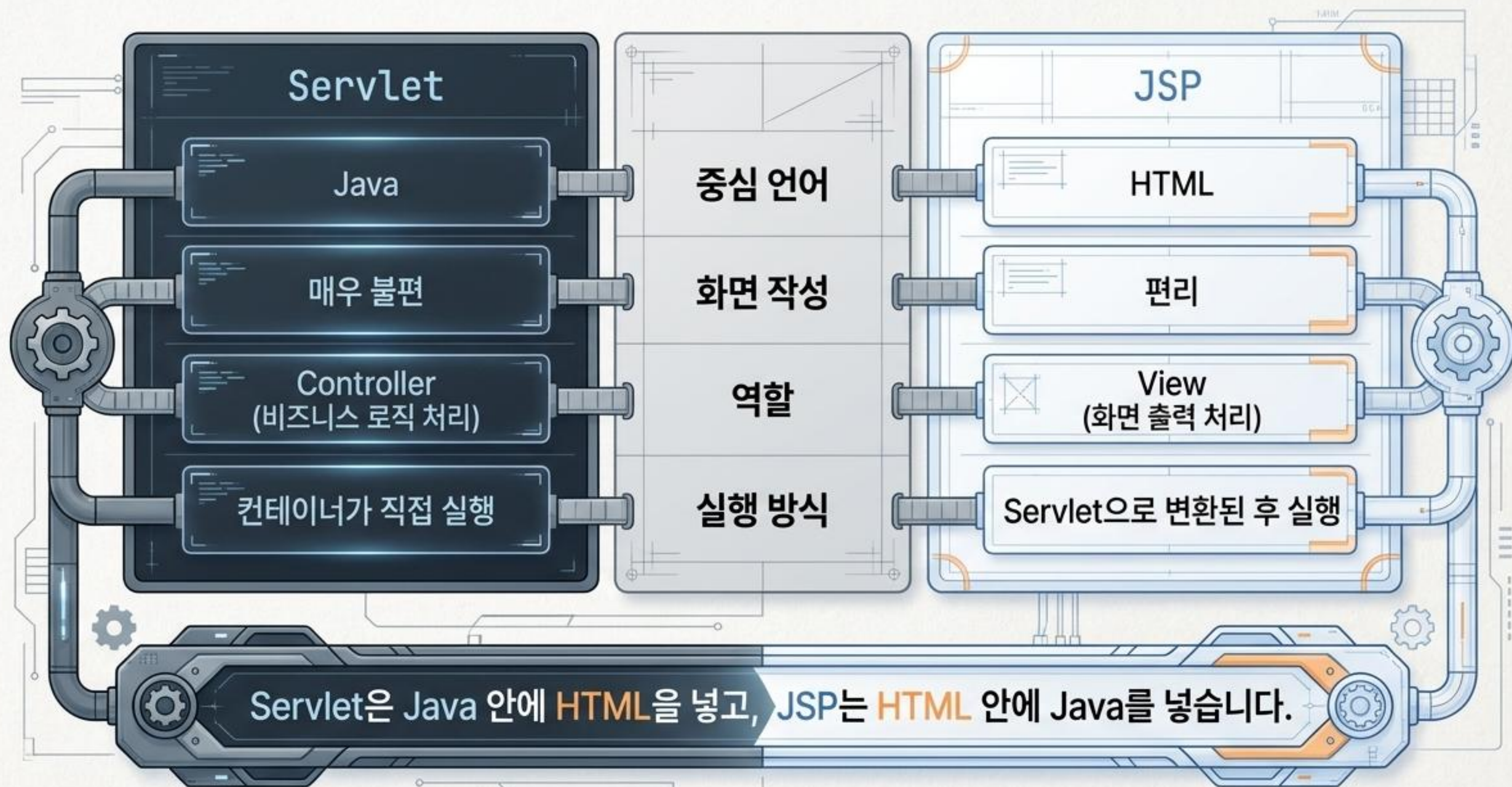
서블릿 생명주기: 싱글톤 객체와 요청별 스레드 할당



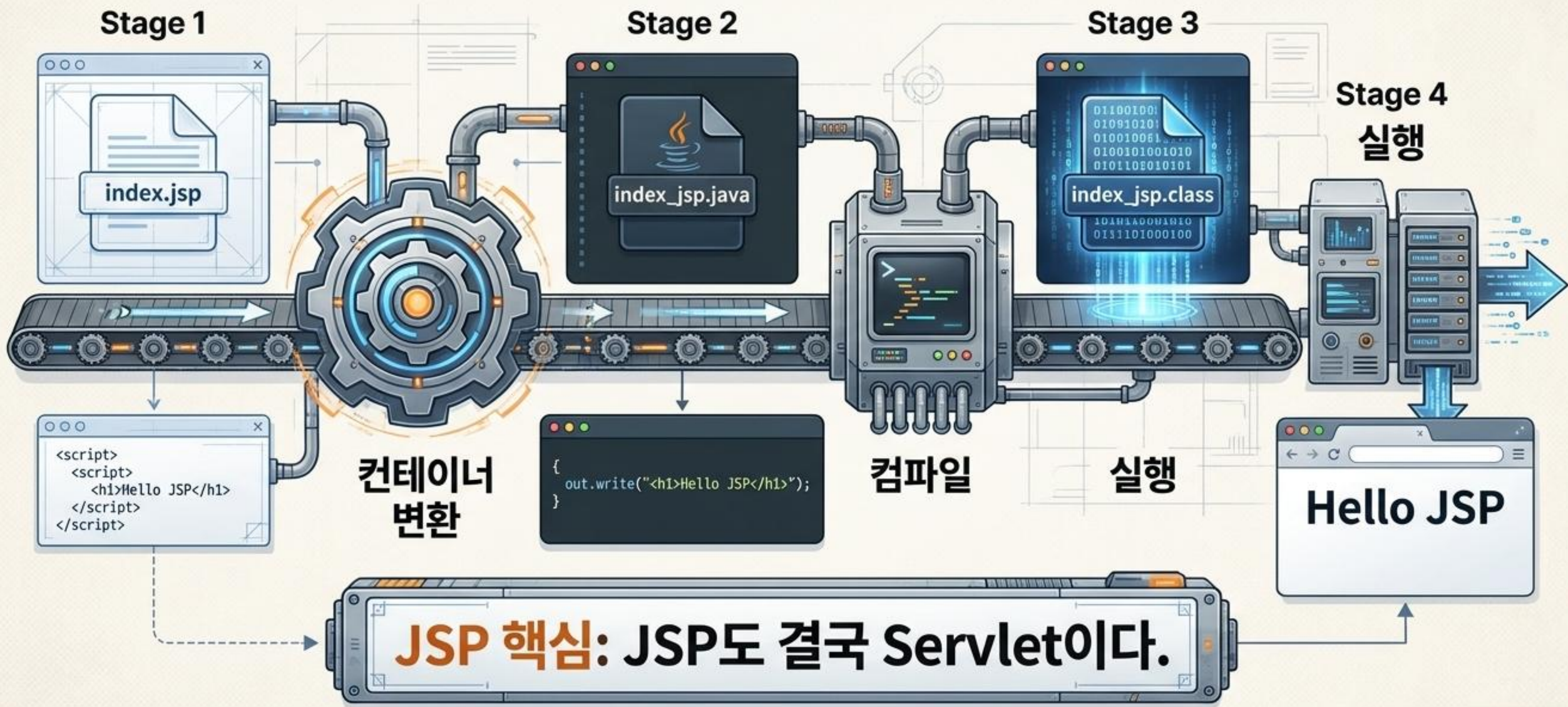
서블릿의 치명적 단점: Java 엔진에 억지로 끼워 넣은 HTML



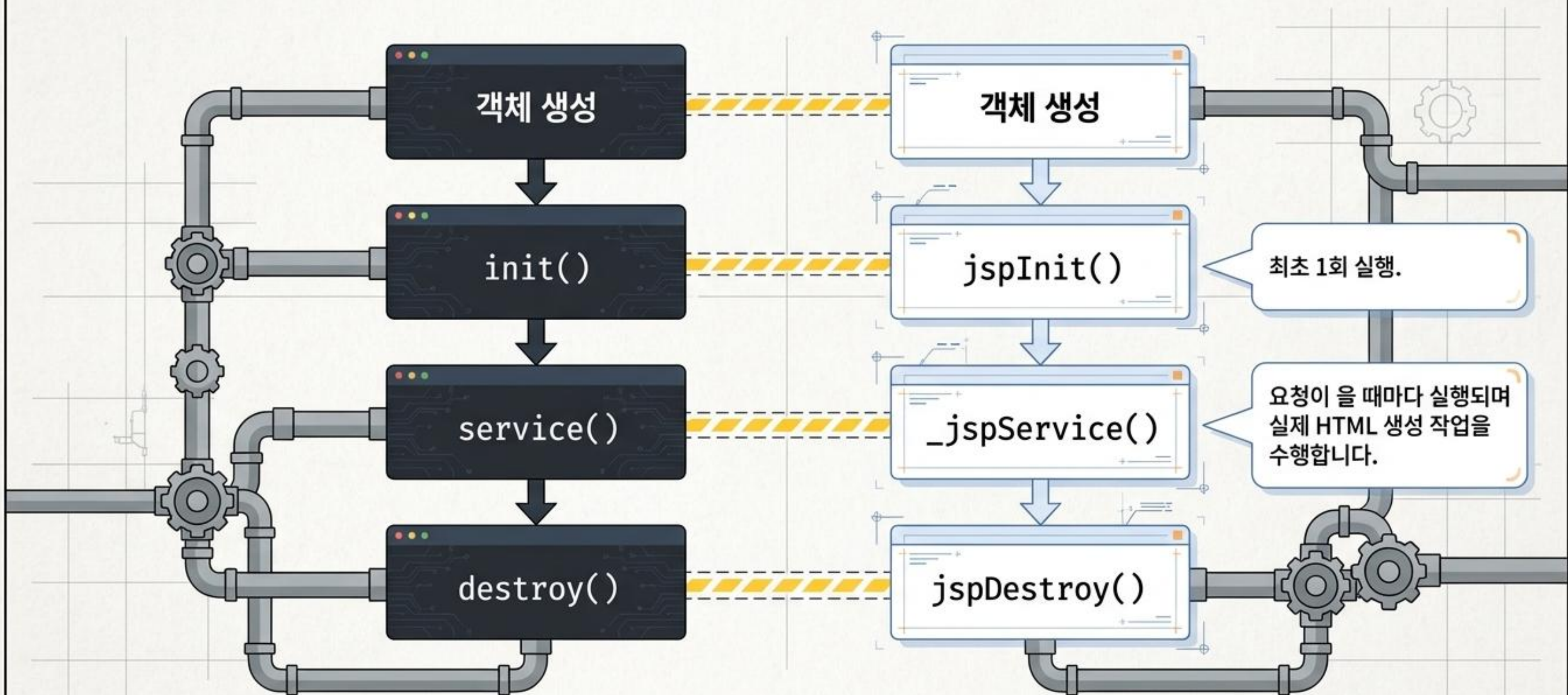
화면 생성을 위한 패러다임의 전환: HTML 중심의 JSP



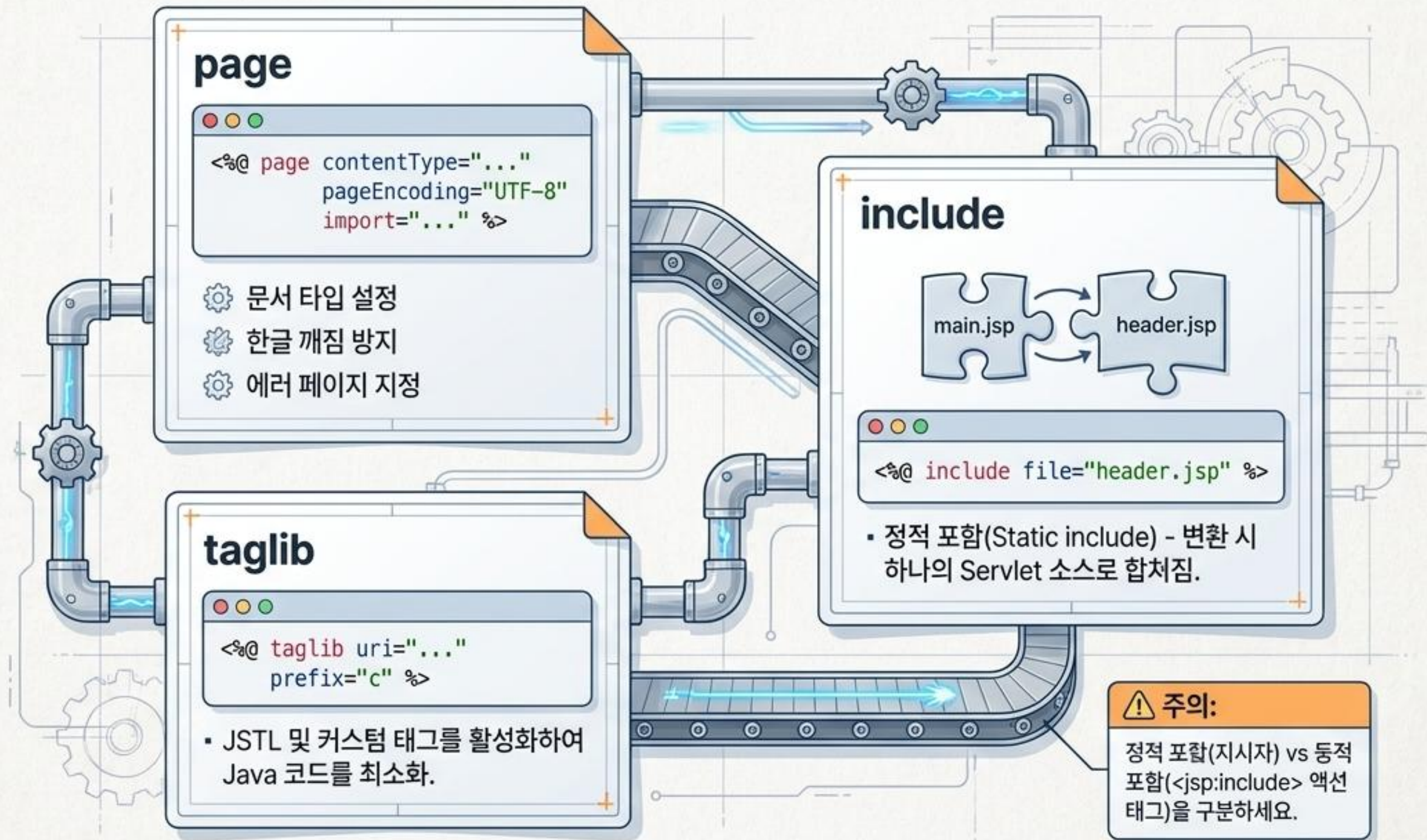
JSP의 실체: 내부적으로는 결국 서블릿으로 번역된다



생명주기 평행이론: Servlet Lifecycle vs JSP Lifecycle



설계도 설정 지시자 (Directives): 페이지의 환경을 정의하다



스크립트 요소의 진화: 하드코딩에서 JSTL/EL로

과거

스크립트 요소	기능 및 설명
<% %> (스크립트릿)	Java 코드 실행
<%= %> (표현식)	값 출력 (out.print()로 변환)
<%! %> (선언문)	멤버 변수 및 메서드 선언

```
<% for(int i=1; i<=5; i++){ %>  
    <%= i %>  
<% } %>
```

MVC 위반

현재

JSTL & EL (선언적 템플릿)

```
<c:forEach var="i" begin="1" end="5">  
    ${i}  
</c:forEach>
```

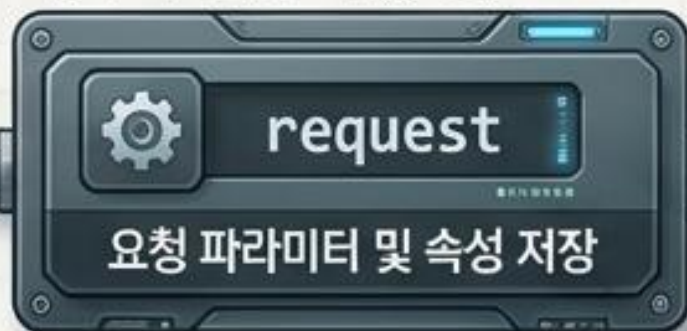
```
<c:forEach var="i" begin="1" end="5">  
    ${i}  
</c:forEach>
```



현재 실무에서는 스크립트릿 사용을 지양하고, 가독성과 유지보수를 위해 JSTL과 EL을 사용합니다.

컨테이너가 제공하는 기본 장비: 9가지 내장객체(Implicit Object)

데이터 입출력 (I/O)



상태 및 공유 (State)

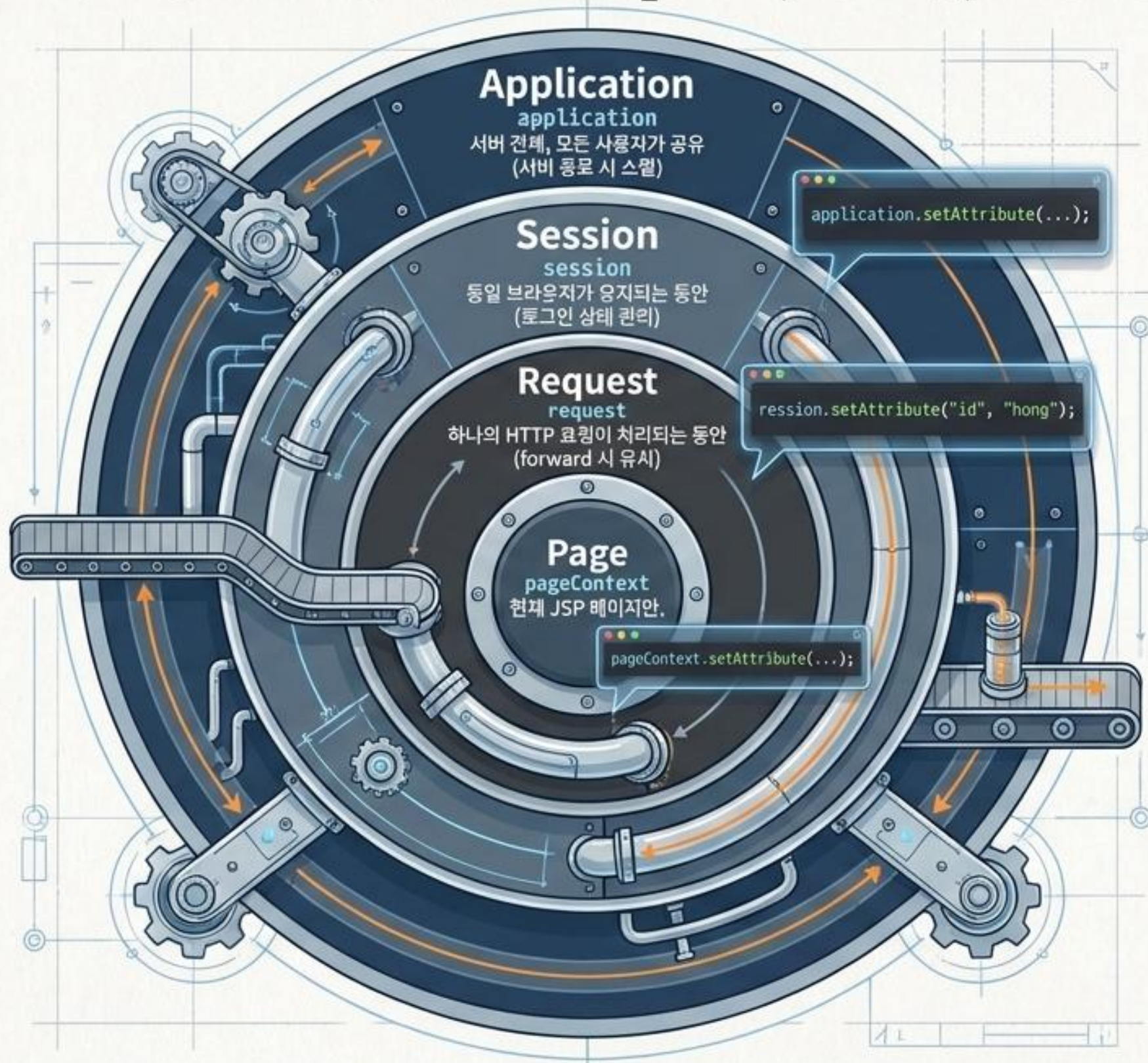


설정 및 제어 (Config/Control)



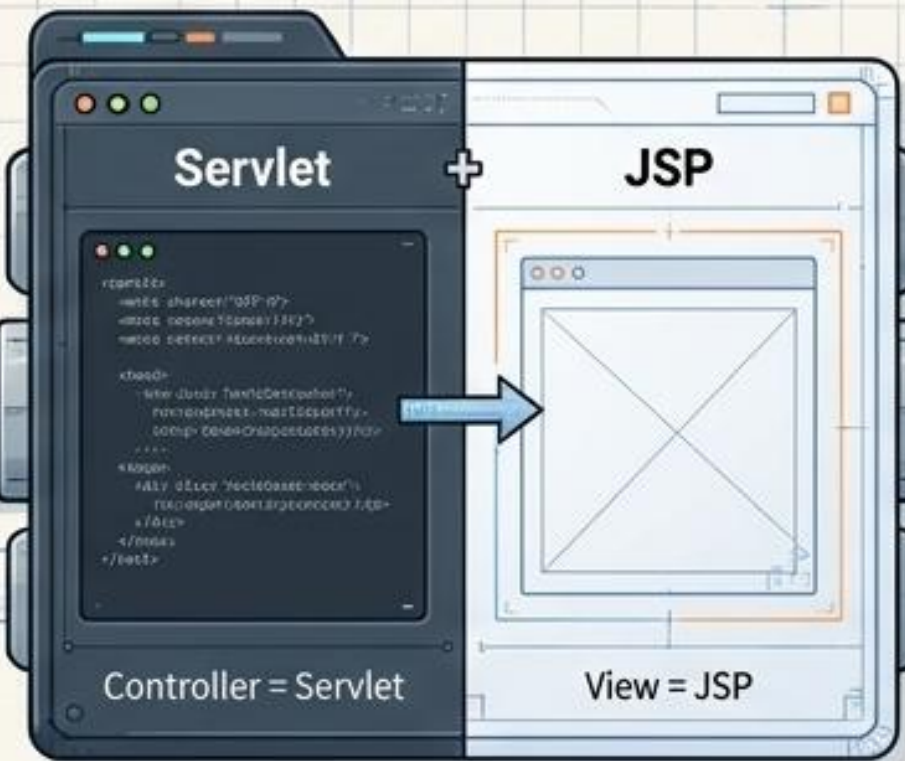
JSP가 객체를 자동 제공하는 이유 : JSP는 내부적으로 Servlet이므로 Servlet API 객체들을 별도 생성 없이 컨테이너로부터 바로 전달받아 사용할 수 있습니다.

데이터의 생존 주기: 4가지 Scope (영역) 아키텍처



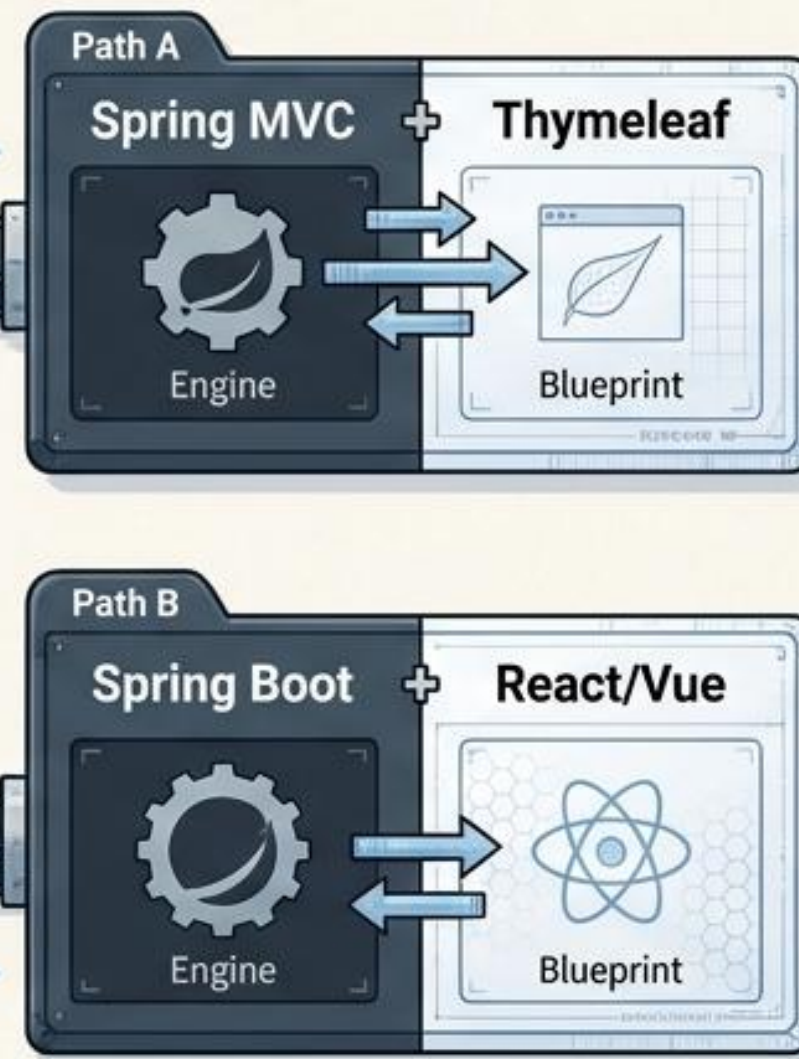
레거시에서 모던 웹으로: 현재 실무에서의 위치

초기 웹 개발



엄격한 MVC 분리.
Controller = Servlet, View = JSP.

현재의 주류



JSP는 끝났는가?

사용량은 감소했지만
공공기관,
엔터프라이즈 SI 등
레거시 시스템
유지보수를 위해
여전히 핵심적인
지식입니다.

Servlet과 JSP의 내부 메커니즘을 이해하는 것은 Spring과 같은
모던 프레임워크의 동작 원리를 마스터하는 필수적인 기초입니다.

동적 웹의 설계도: JSP와 엔진의 작동 원리

레거시 엔터프라이즈 시스템부터 Spring Boot 통합까지, 웹 애플리케이션 서버를 구동하는 하이브리드 기술의 이해.

```
<div class="container">
  <meta character="utf-8">
  <link rel="stylesheet" href="styles.css">
</div>
<body>
  <meta class="ist" />
</div>
</div>
```

HTML
(구조 및 프레젠테이션)

```
<div class="container">
  <div context="container">
    <link rel="stylesheet" href="styles.css">
  </div>
</div>
</div>
```

JSP (JavaServer Pages)

```
public class ServerLogic {
  ...
  public data data {
    Server(data = dataStaaco());
    System.out.println(data);
  }
}
```

Java
(비즈니스 로직 및 서버)

```
public class ServerLogic {
  System.out.println(data);
  // and data
  System.out.println(data);
}
```

정적 HTML의 한계와 서블릿(Servlet)의 딜레마

초기 웹은 사용자 맞춤형 동적 콘텐츠를 제공할 수 없는 정적 HTML 문서에 불과했습니다. 이를 극복하기 위해 Java 코드로 웹을 제어하는 Servlet이 등장했지만, 코드의 가독성과 유지보수성에 치명적인 문제가 있었습니다.

The Problem - Servlet

```
out.println("<html>");  
out.println("<body>");  
out.println("<h1>환영합니다!</h1>");  
out.println("</body>");  
out.println("</html>");
```

Java 코드 안에 HTML 문자열 삽입 (가독성 저하, 수정 불편)

The Solution - JSP

```
<html>  
  <body>  
    <% int a = 10; %>  
    <h1> <% String user = "홍길동"; %>  
      <%= user %>님, 환영합니다! %> </h1>  
    <% if (a > 5) {  
      out.println("A는 5보다 큽니다.");  
    } %>  
  </body>  
</html>
```

HTML 문서 내부에 Java 코드 삽입 (직관적인 UI 설계, 쉬운 유지보수)

실행 공간의 분리: JSP vs JavaScript

구분	JSP (JavaServer Pages)	JavaScript (JS)
실행 위치	 서버 (Server) 측: WAS 내부에서 실행	 클라이언트 (Browser) 측: 사용자의 기기에서 실행
주요 역할	DB 연동, 비즈니스 로직 처리, 보안 데이터 핸들링	화면 이벤트 처리, UI 제어, 폼 유효성 검사
페이지 동작	서버 통신 및 동적 페이지 렌더링 (새로고침 발생)	브라우저 내에서 즉각적인 동적 변화 (새로고침 없음)

JSP는 결과를 HTML로 변환하여 브라우저에 '전달'할 뿐,
브라우저 위에서 직접 실행되지 않습니다.

언더 더 후드: JSP 동작의 6단계 라이프사이클

JSP는 최종적으로 Servlet으로 변환되어 실행됩니다.
최초 요청 시에만 컴파일 과정을 거치며, 이후에는
변환된 서블릿을 재사용하여 빠르게 응답합니다.

1. 요청 (Request)
클라이언트가 hello.jsp를
WAS에 요청.

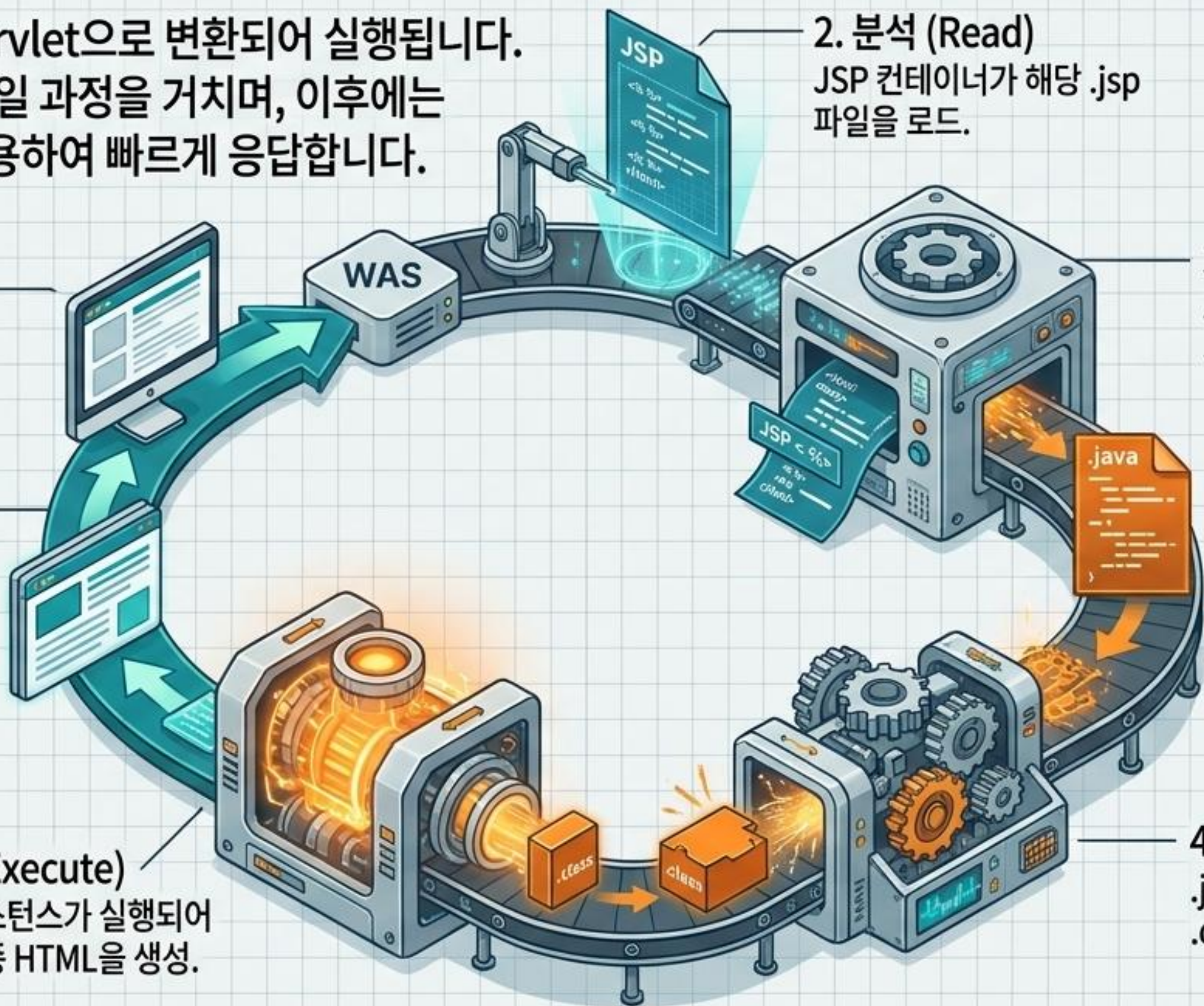
6. 응답 (Response)
완성된 HTML을
HTTP 프로토콜을 통해
클라이언트에 전달.

5. 실행 (Execute)
서블릿 인스턴스가 실행되어
동적인 최종 HTML을 생성.

2. 분석 (Read)
JSP 컨테이너가 해당 .jsp
파일을 로드.

3. 변환 (Generate)
JSP 코드를 분해하여 자바
서블릿 .java 파일로 변환.

4. 컴파일 (Compile)
.java 파일을 실행 가능한
.class 파일로 컴파일.



JSP 개발의 핵심 도구 상자 (The JSP Toolkit)



지시어 (Directive)

```
<%@ ... %>
```

설정 및 포함

페이지 속성, 라이브러리 설정,
외부 파일 포함.



스크립트 (Script)

```
<% ... %>
```

자바 로직 삽입

변수 선언, 연산, 메서드 정의 등
직접적인 Java 코드 실행.

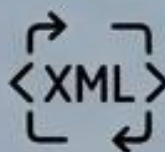


내장 객체 (Implicit Objects)

```
request, response
```

사전 정의된 서버 자원

별도 선언 없이 즉시 사용 가능한
웹 통신 객체.



액션 태그 (Action Tag)

```
<jsp:include>
```

XML 기반 서버 제어

자바 빈(JavaBean) 연동 및
페이지 흐름 제어.



표현 언어 (EL)

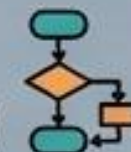
```
${expr}
```

데이터 바인딩

복잡한 자바 코드 없이 데이터를
간결하게 출력하는 문법.



JSTL (표준 태그 라이브러리)



```
<c:if>
```

로직의 태그화

조건문, 반복문 등 비즈니스 로직을
HTML 친화적 태그로 캡슐화.

기반 설정과 로직 실행: 지시어와 스크립트

지시어는 페이지의 메타데이터를 설정하
설정지 속성과 스크립트나.
스크립트서 동적적인 데이터를 만적을
데이터를 만들합니다. 쥔 적한색상에
코상해 콧딩다.

지시어 (Directive) - 페이지의 메타데이터 정의

page: 스크립트 언어, 콘텐츠 타입 지정.

```
<%@ page contentType="text/html; charset=UTF-8"  
import="java.util.Date" %>
```

include: 번역 단계에서 외부 파일 병합.

```
<%@ include file="header.jsp" %>
```

taglib: 태그 라이브러리 선언.

```
<%@ taglib uri="..." prefix="c" %>
```

스크립트 (Script) - 동적 데이터 생성

선언 (Declaration): 메서드 및 전역 변수 정의

```
<%! public String greet(String name) { ... } %>
```

스크립트릿 (Scriptlet): 일반 Java 실행 코드

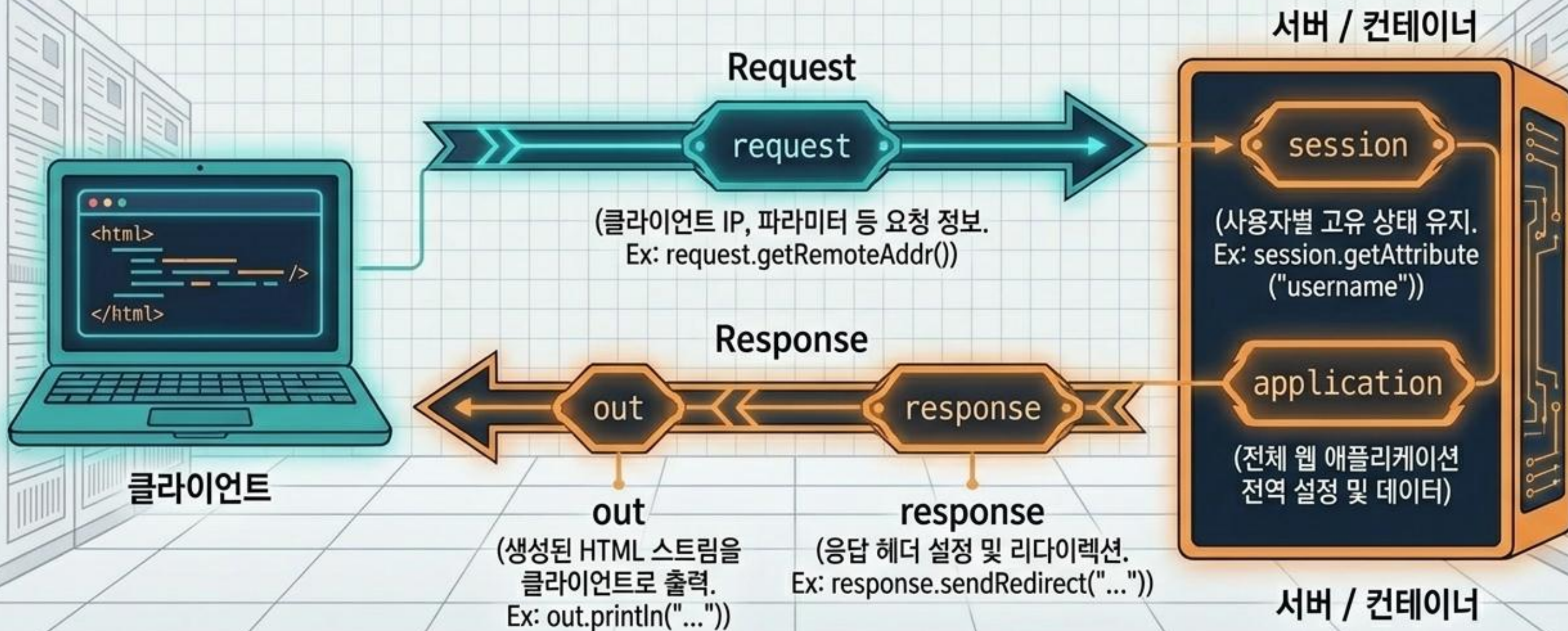
```
<% int a = 10; int b = 20; int sum = a + b; %>
```

표현식 (Expression): 결과값을 HTML에 출력

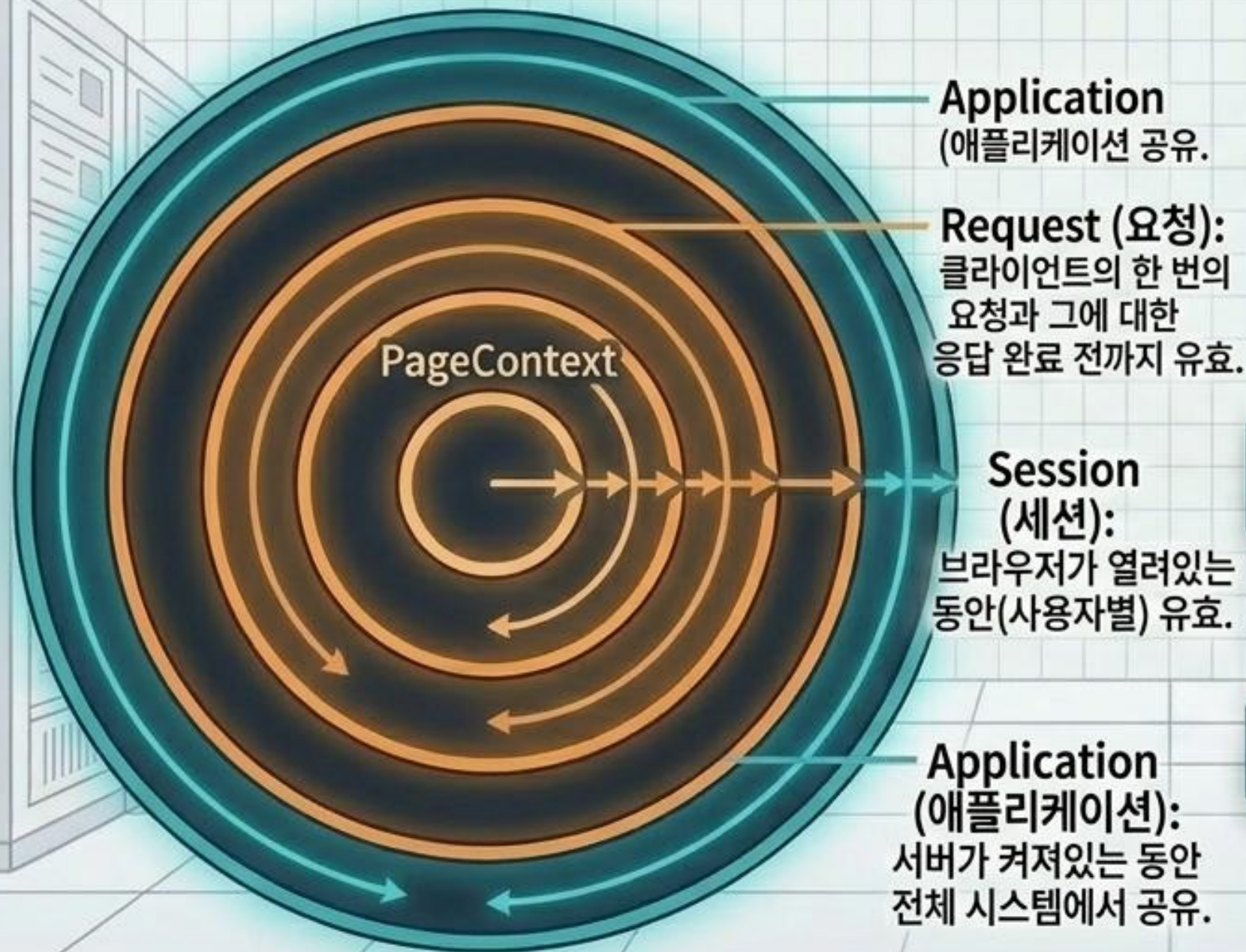
```
<p>합계: <%= sum %></p>
```

보이지 않는 조력자: 내장 객체 (Implicit Objects)

개발자가 명시적으로 선언할 필요 없이, JSP 컨테이너가 자동으로 생성하여 제공하는 사전 정의된 자바 객체들입니다.



데이터 탐색의 확장: EL (표현 언어)과 스코프(Scope)



EL(Expression Language)은 스크립트없이 데이터를 직관적으로 출력하는 `${...}` 형태의 언어입니다. 변수 호출 시 지정된 스코프(Scope) 계층을 따라 안쪽에서 바깥쪽으로 데이터를 자동 탐색합니다.

```
example.jsp?username=John 요청 시:
${username}
${param.username}
...
```

복잡한 Java 코드 없이 스코프를 탐색하여 즉시 'John' 출력 가능.

코드의 진화: 스크립트릿에서 JSTL / 액션 태그로

JSP 내부에 Java 코드를 무분별하게 섞어 쓰면 가독성과 유지보수성이 파괴됩니다.
액션 태그와 JSTL을 사용하면 로직을 XML 형태의 태그로 캡슐화하여 뷰(View) 코드를 깔끔하게 유지할 수 있습니다.

⚠ Anti-Pattern: 스크립트릿

```
<html>
<%
    String targetPage = "profile.jsp";
    request.setAttribute("userId", "12345");
    request.setAttribute("userRole", "admin");
    RequestDispatcher dispatcher =
        request.getRequestDispatcher(targetPage);
    dispatcher.forward(request, response);
%>
</body>
</html>
```

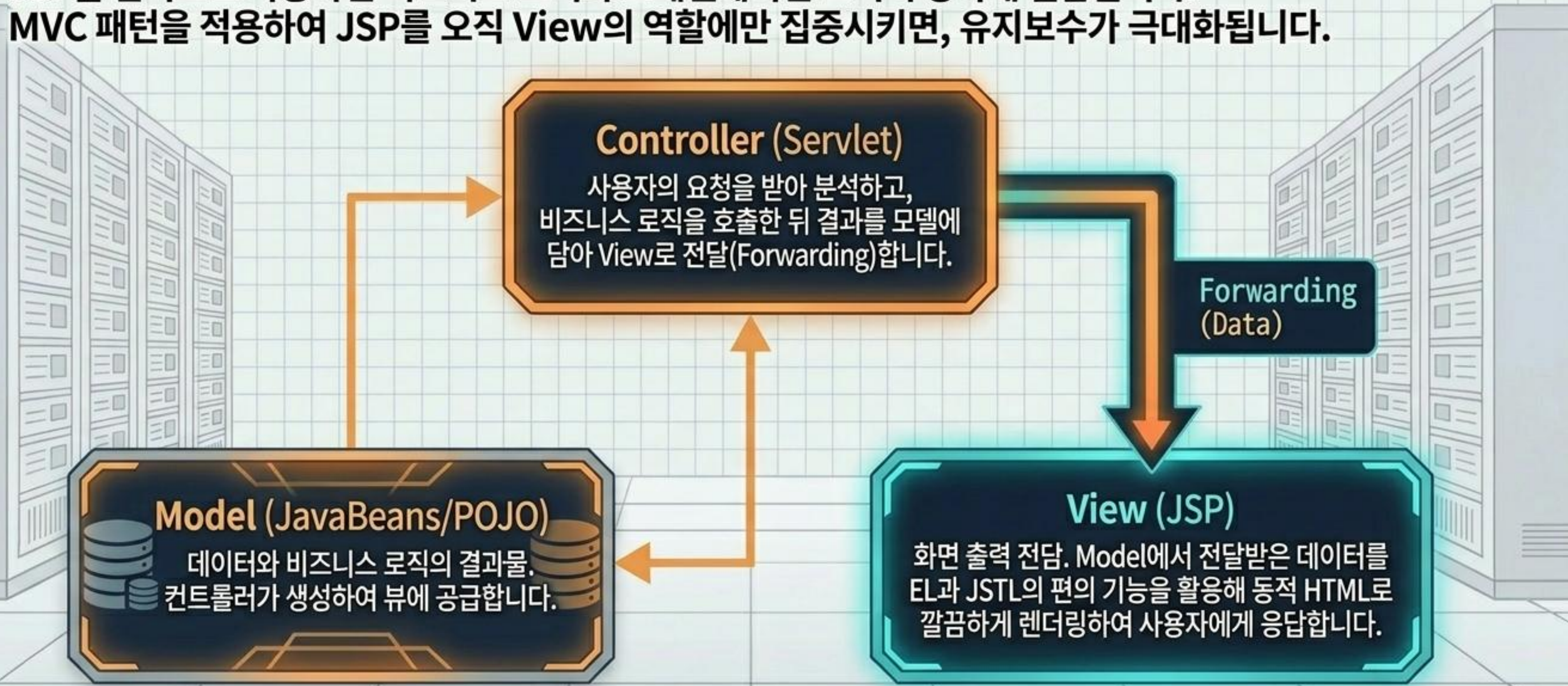
✅ Best Practice: 액션 태그 & JSTL

```
<jsp:forward page="profile.jsp">
    <jsp:param name="userId"
        value="12345" />
    <jsp:param name="userRole"
        value="admin" />
</jsp:forward>
```

JSTL 활용: <c:if test="\${userRole == 'admin'}">
처럼 제어문(if, for)까지 태그화하여 완전한 로직 분리 가능.

역할의 완벽한 분리: JSP와 MVC 아키텍처

JSP를 단독으로 사용하면 비즈니스 로직과 프레젠테이션 로직이 강하게 결합됩니다.
MVC 패턴을 적용하여 JSP를 오직 View의 역할에만 집중시키면, 유지보수가 극대화됩니다.



모던 인프라와의 결합: Spring Boot 환경의 JSP 설정

Spring Boot의 내장 톰캣은 기본적으로 JSP 렌더링 엔진을 포함하지 않습니다.
따라서 별도의 의존성 주입과 경로 라우팅 설정이 필수적입니다.





레거시 시스템 운영을 위한 필수적 무기

최신 프레임워크와 템플릿 엔진(Thymeleaf, React 등)의 발전에도 불구하고, JSP는 여전히 가치를 지닙니다.



방대한 엔터프라이즈 레거시 //

국내 금융, 공공, 대기업의 수많은 기존 시스템이 JSP와 Model 1/2 아키텍처를 기반으로 구축되어 있습니다.



안정적인 뷰 렌더링 //

비즈니스 로직과 프레젠테이션 로직을 분리하는 MVC 패턴의 철학을 이해하는 가장 직관적인 기반 기술입니다.



유지보수의 핵심 //

기존 코드를 읽고, EL/JSTL을 활용해 안전하게 리팩토링하며 프레임워크의 전환을 준비하기 위해 JSP 구조에 대한 깊은 이해는 필수적입니다.

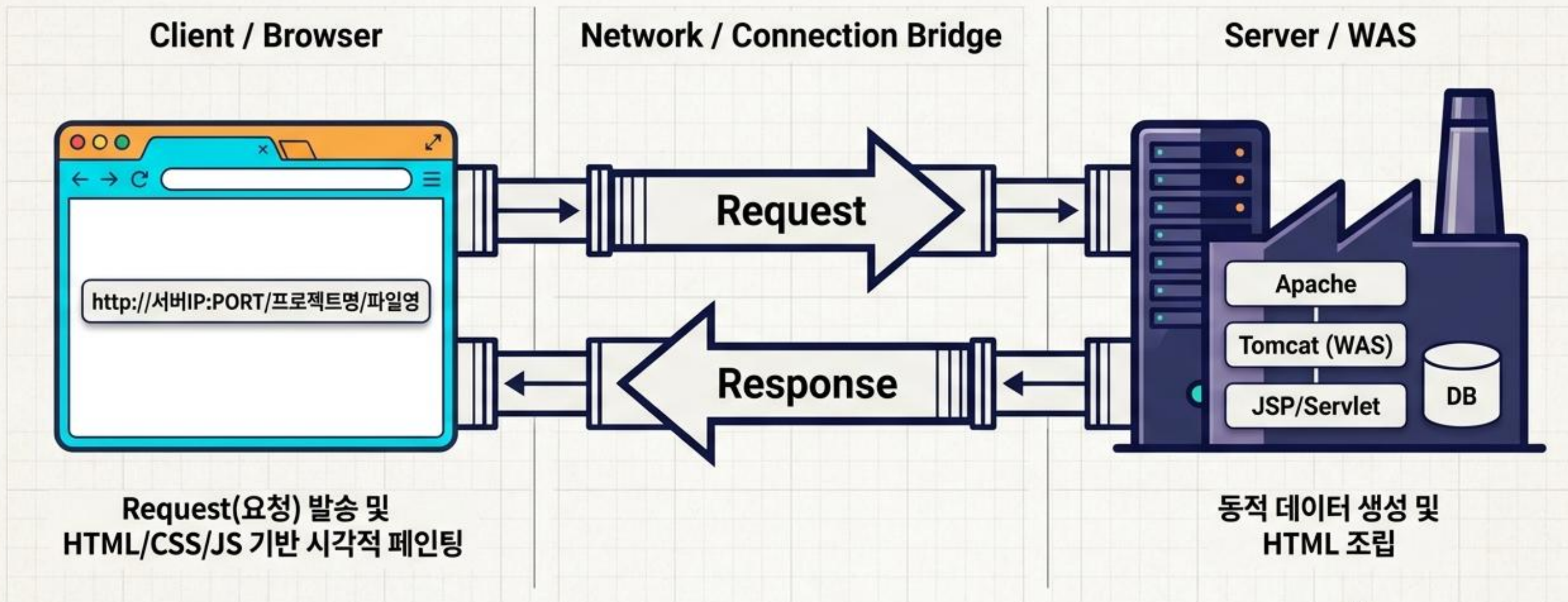
Servlet의 복잡성을 넘어, 동적 웹의 뼈대를 세우다.

웹 애플리케이션의 청사진

JSP 백엔드 엔진부터 CSS 프론트엔드 스타일링까지의 완벽한 조립 가이드



풀스택 생태계 맵 (The Full-Stack Ecosystem)



JSP는 보이지 않는 곳에서 동적 데이터를 조립하는 엔진이며,
CSS는 브라우저에 도달한 뼈대를 아름답게 배치하는 외관입니다.

뷰(View) 엔진의 진화: 왜 Servlet 대신 JSP인가?

Servlet - The Problem

순수 Java 영역 (HTML 작성의 고통)

```
out.write("<html>");  
out.write("<body>");  
out.write("<h1>Hello Servlet!!</h1>");
```

보안과 비즈니스 로직 처리에는 탁월하지만,
UI 코드를 Java 문자열로 한 줄씩 작성해야 하므로
화면 수정 시 매번 재컴파일이 필요.

JSP - The Solution

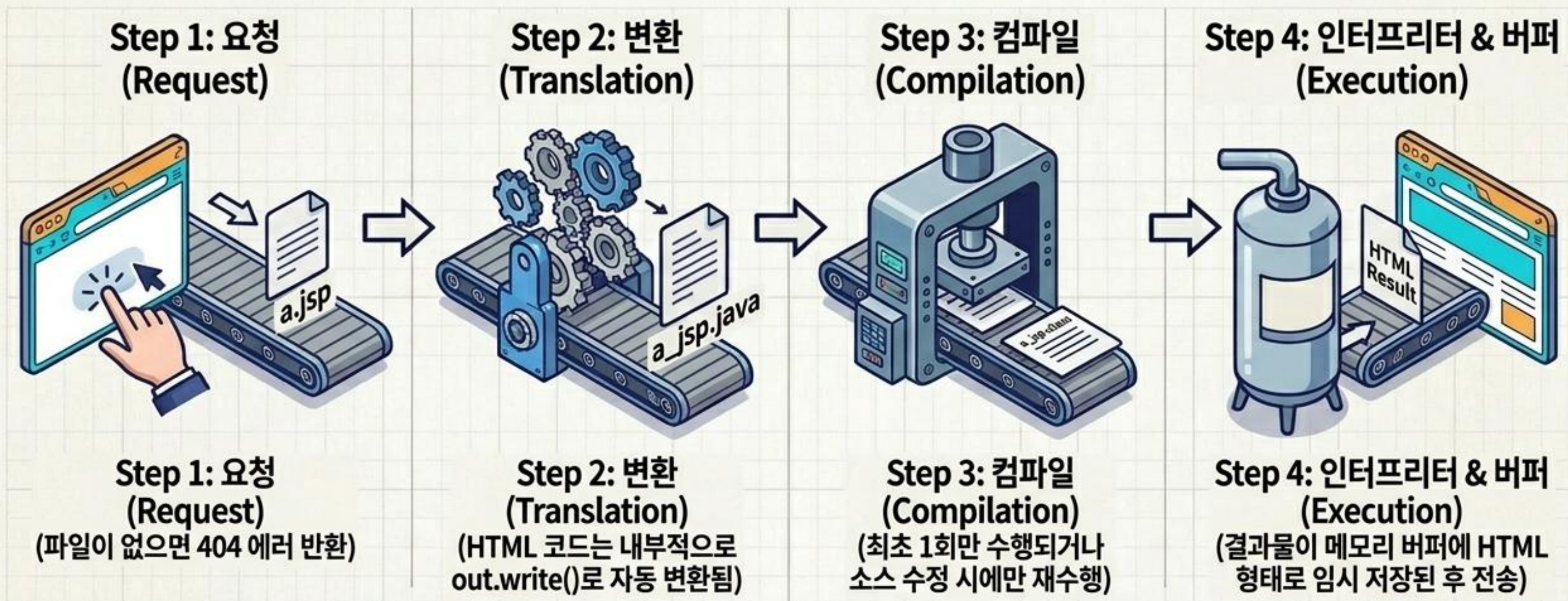
HTML 내장 영역 (Front-End 친화적)

```
<h1><%= msg %></h1>
```

HTML 골격 안에 Java 코드를 필요한 곳에만
블록 조립하듯 삽입. 수정 시 Tomcat이
자동 컴파일하므로 생산성 극대화.

JSP는 HTML의 편리함과 Java의 동적 데이터 처리 능력을 결합한 최적의 뷰(View) 템플릿입니다.

보이지 않는 톱니바퀴: JSP 컴파일 라이프사이클



**JSP는 결국 서블릿(Servlet)입니다.
톰캣이 귀찮은 `out.write()` 변환 작업을 개발자 대신 수행해 줄 뿐입니다.**

구문 해부도 (Syntax Anatomy): JSP는 자바로 어떻게 번역되는가?

JSP 원본 코드

```
<%! int a=10;  
public void display(){} %>
```

Snippet 1

전역 영역으로 이동.
문장 끝에 ; 필수.

```
<% String name="홍길동"; %>
```

Snippet 2

지역 변수 및
일반 로직 삽입.
문장 끝에 ; 필수.

```
<%= name %>
```

Snippet 3

out.println()으로
자동 변환.
; 생략.

번역된 서블릿 코드

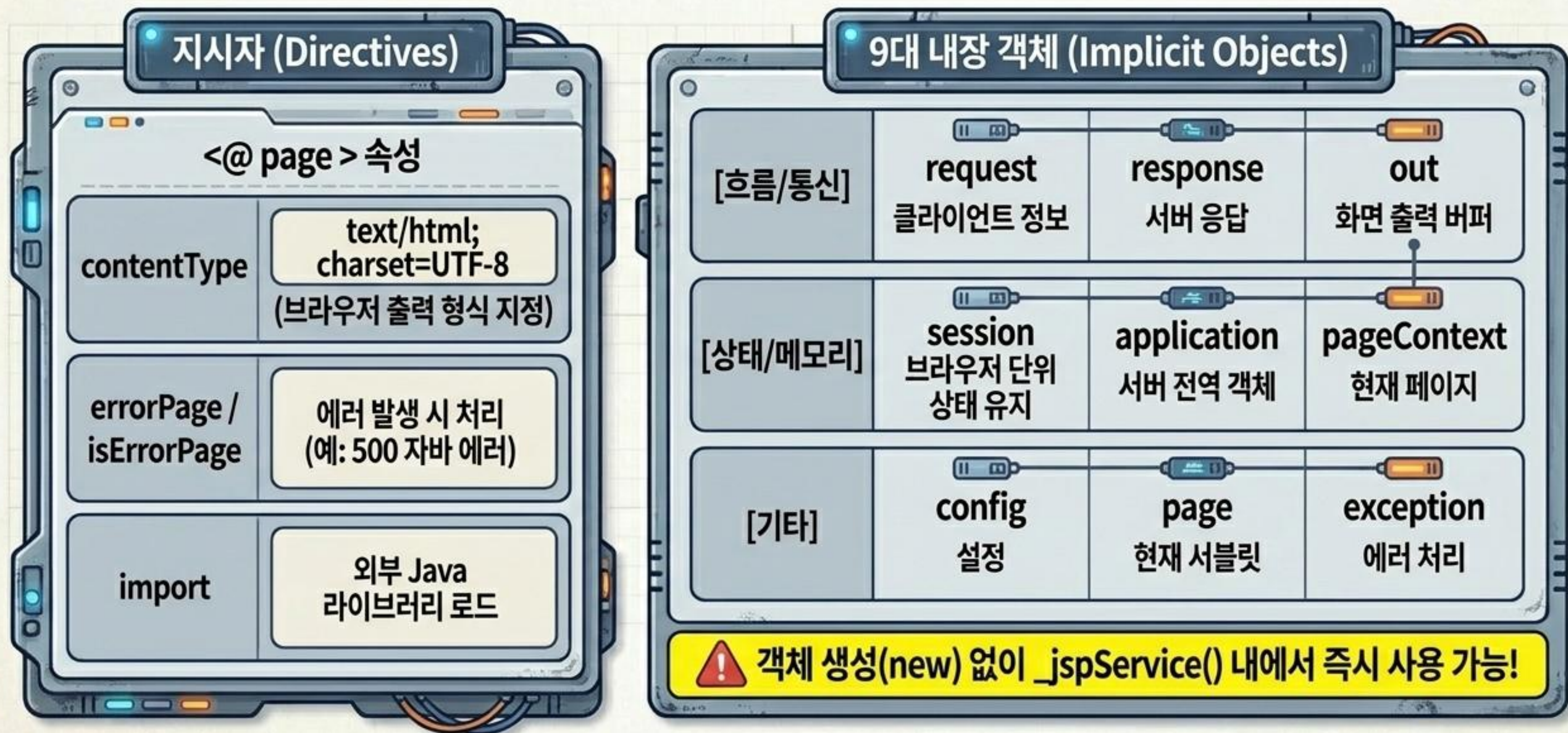
멤버 변수 (전역) 및 메서드 영역

_jspService() 내부

```
out.println(name);
```

번역된 서블릿 클래스 코드

컨트롤 패널: 지시자와 9대 내장 객체



모던 뷰(View) 템플릿: JSTL과 EL로의 전환

과거 (Legacy): 스크립트릿 방식 - 유지보수 지옥

```
<% for(int i=2; i<=9; i++) { %>
    <th><%= i+"단" %></th>
<% } %>
```

HTML 꺾쇠와 Java의
중괄호가 뒤섞여 가독성 저하.
(MVC 패턴 위배)

현재 (Modern): JSTL & EL 방식 - 클린 코드

```
<c:forEach var="i" begin="2" end="9">
    <th>${i}단</th>
</c:forEach>
```

완벽한 태그형 문법 구조.
\${} (EL)를 통한 객체
직관적 접근.

Tip: <jsp:useBean> 및 <jsp:setProperty>와 같은 액션 태그를 통해 View 계층에서
복잡한 Java 인스턴스화를 완전히 제거할 수 있습니다.



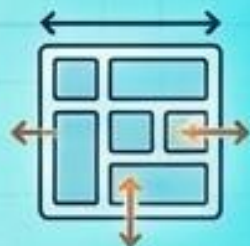
서버에서의 **동적 데이터 조립**이 완료되었습니다.
이제 이 뼈대를 **브라우저**라는 **캔버스** 위에서 어떻게 **배치**하고 **꾸밀** 것인가?



- 의미론적 뼈대 세우기
(Semantic HTML)



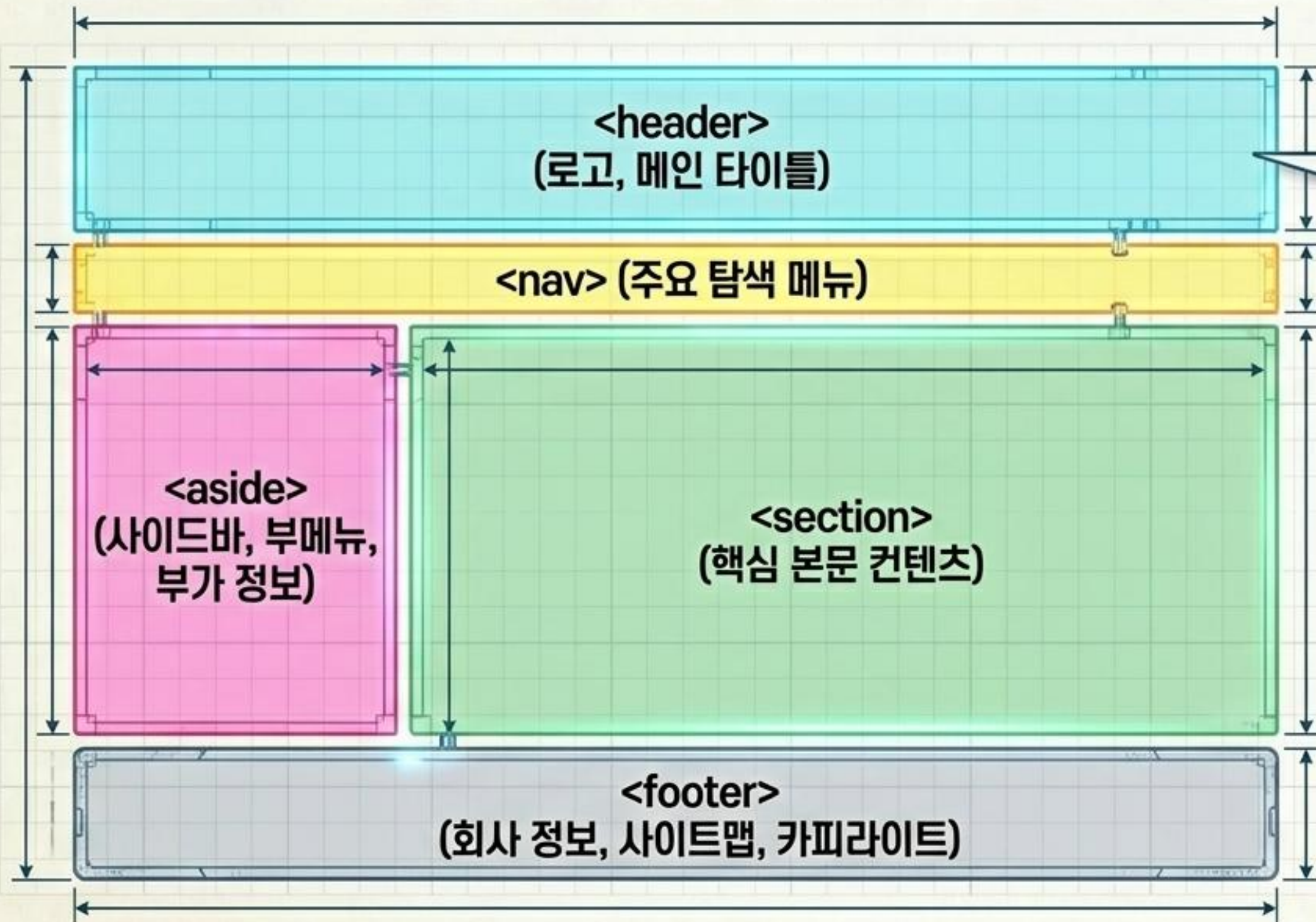
- 2D/3D 공간에 요소 배치하기
(Position & Z-Index)



- 레이아웃의 유동성 제어하기
(Flexbox & Grid)

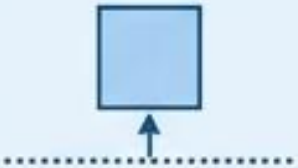
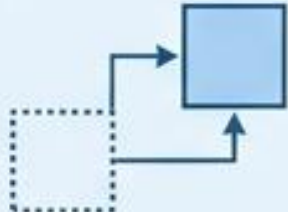
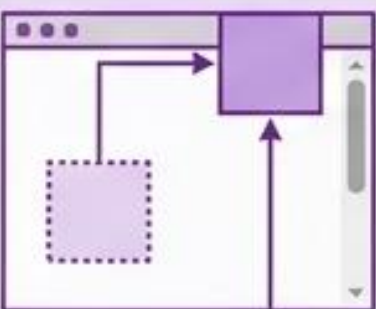


시멘틱 와이어프레임 (Semantic HTML): 'div 지옥' 탈출하기



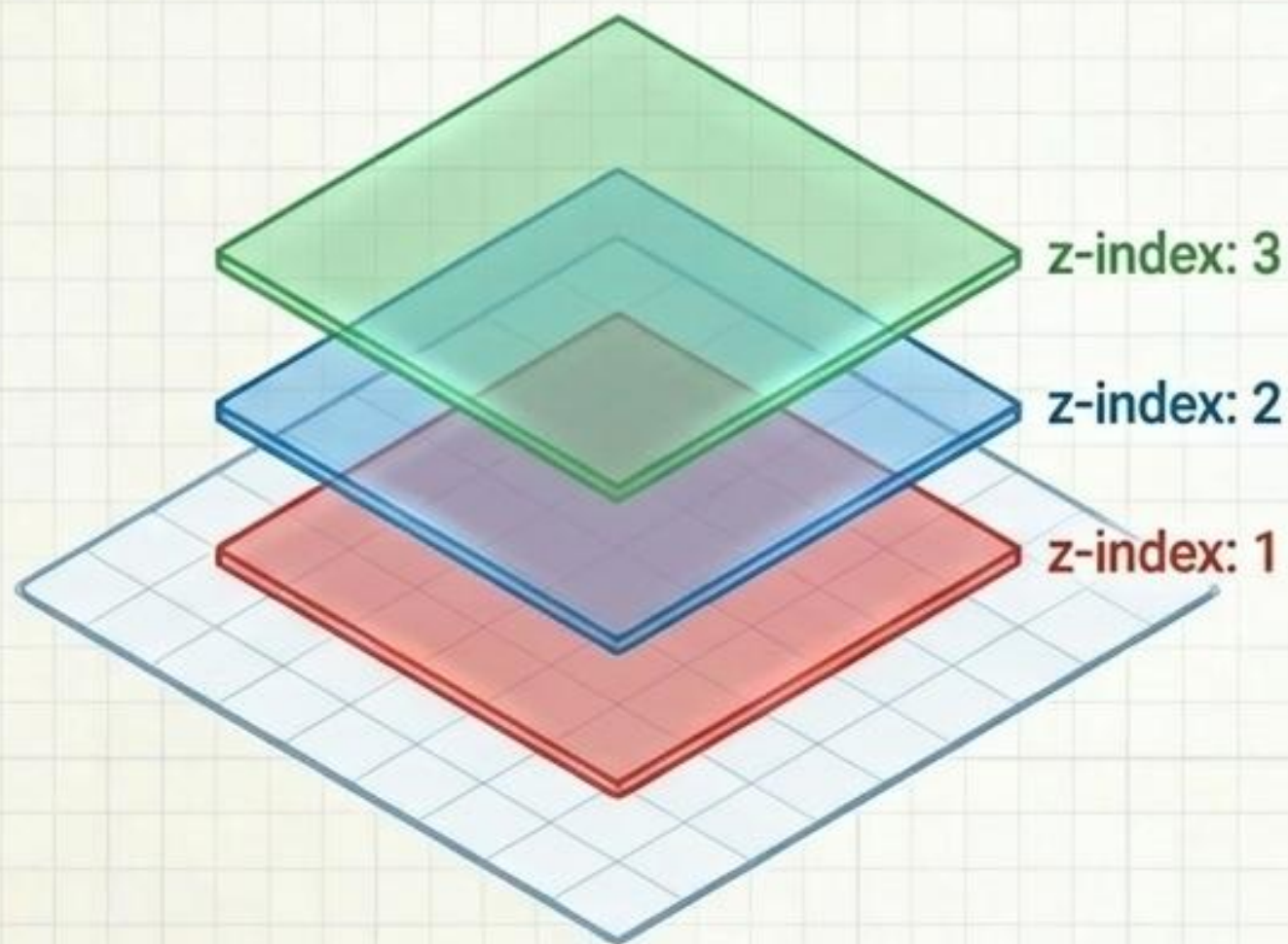
`<div id="header">` 대신
`<header>`를 사용함으로써,
브라우저와 검색엔진(SEO)
모두가 영역의 목적을 명확히
이해할 수 있습니다.

CSS 공간 좌표계 (The Positioning Matrix)

		문서 기본 흐름 (Document Flow)	
		유지 (In Flow) ▽	이탈 (Out of Flow) ▽
기준 좌표 (Reference Point)	요소 자신 (Self)	 static (기본값) 문서 기본 흐름. top/left 무시.	
	부모 요소 (Parent)	 relative 기본 흐름 유지. '원래 자신이 있던 자리'를 기준으로 띄워서 이동. (빈 자리 유지됨)	 absolute 문서 흐름 이탈. position이 지정된 '가장 가까운 조상'을 기준으로 절대 좌표 이동.
	뷰포트 (Viewport)	 sticky relative + fixed 하이브리드. 스크롤 위치에 따라 특정 지점 도달 시 화면 상단에 고정.	 fixed 문서 흐름 이탈. '브라우저 화면' 기준으로 고정 스크롤해도 따라다님.

공간의 확장: Z-Axis와 넘침(Overflow)의 미학

Z-Index



숫자가 높을수록 사용자에게 더 가깝게(위로) 올라옵니다.
단, position 속성이 static이 아닐 때만 작동합니다.

Overflow 처리

긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시

overflow: hidden
(넘친 부분이
잘려 보이지 않음)

긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시

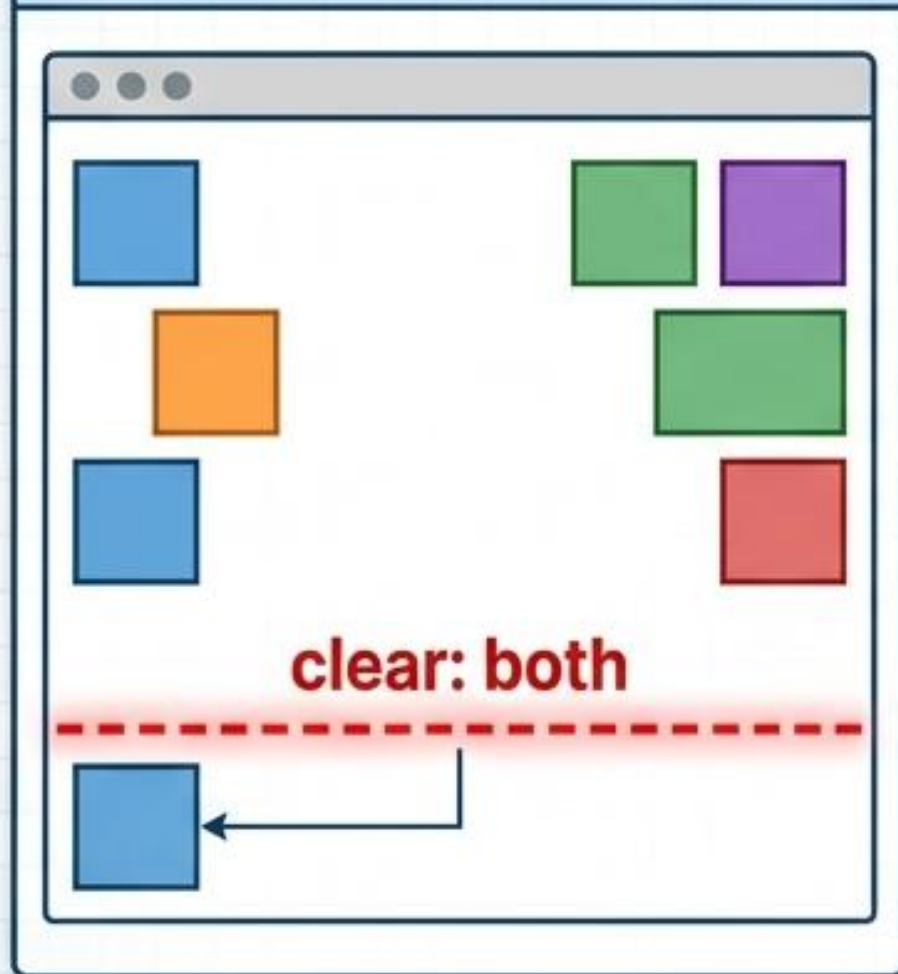
overflow: scroll
(넘침 여부 관계없이
스크롤바 강제 생성)

긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시
긴 텍스트 예시

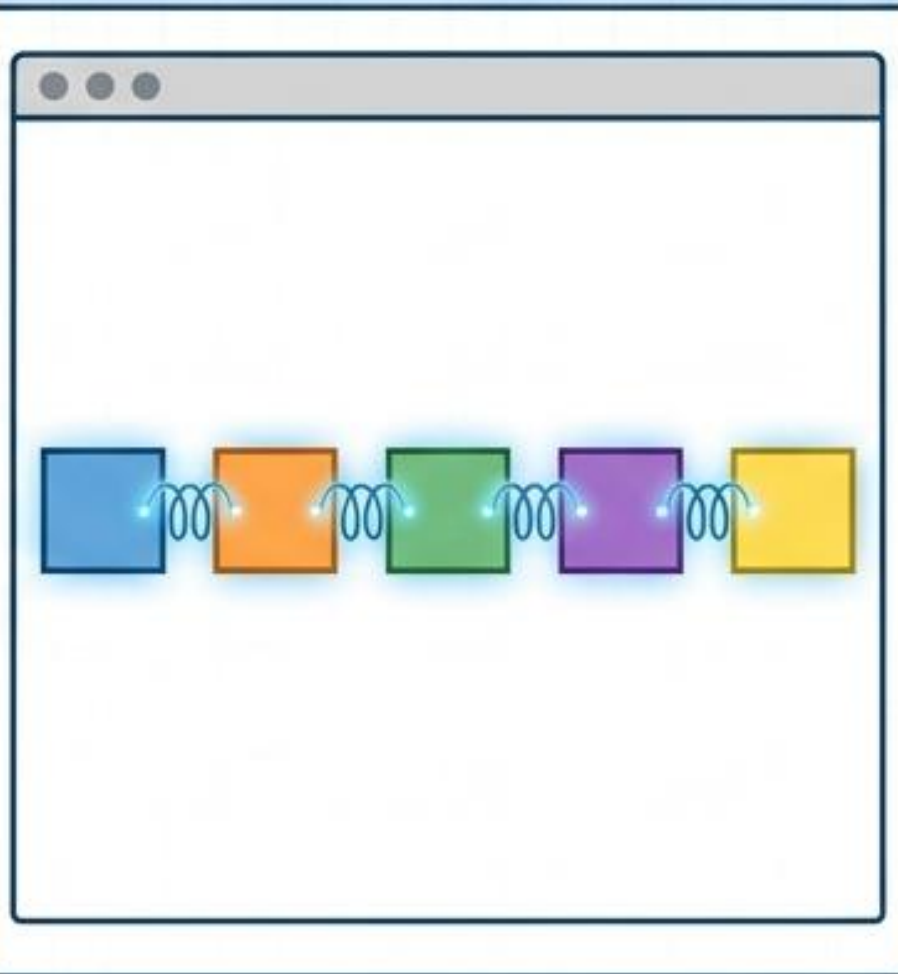
overflow: auto
(넘칠 때만 스마트하게
스크롤바 자동 생성)

흐름과 정렬 (Layout Evolution): Float에서 Grid까지

전통 방식: float & clear

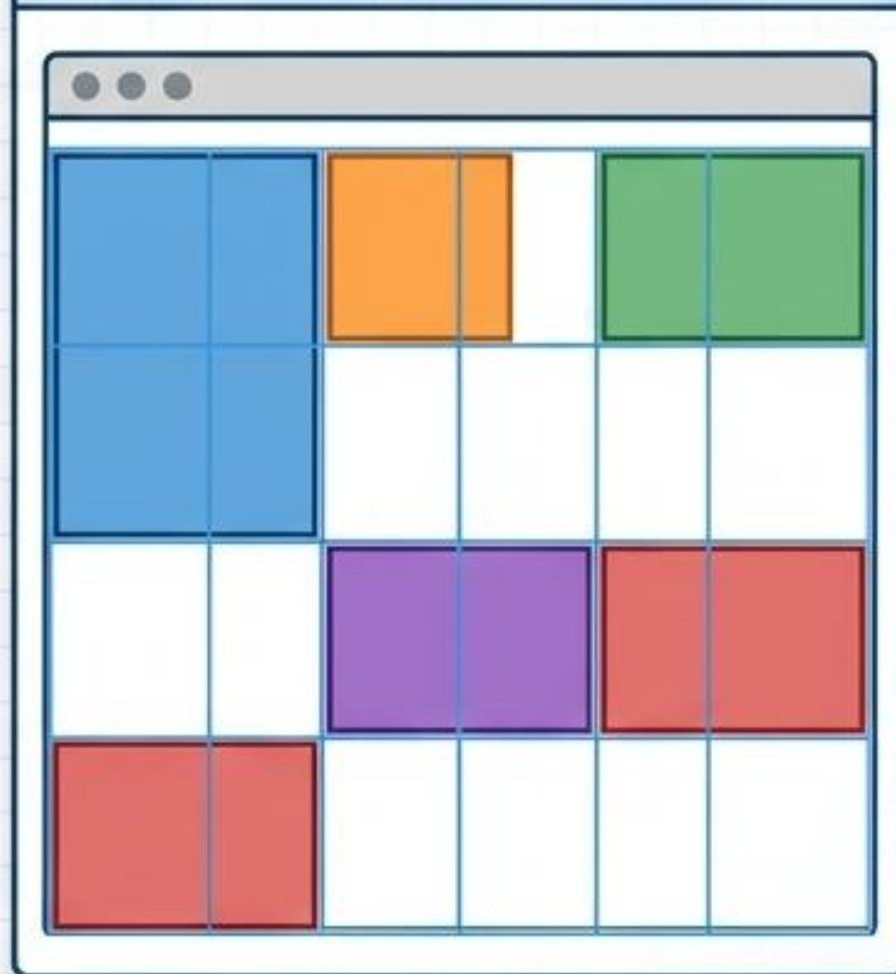


1차원 배치: display: flex



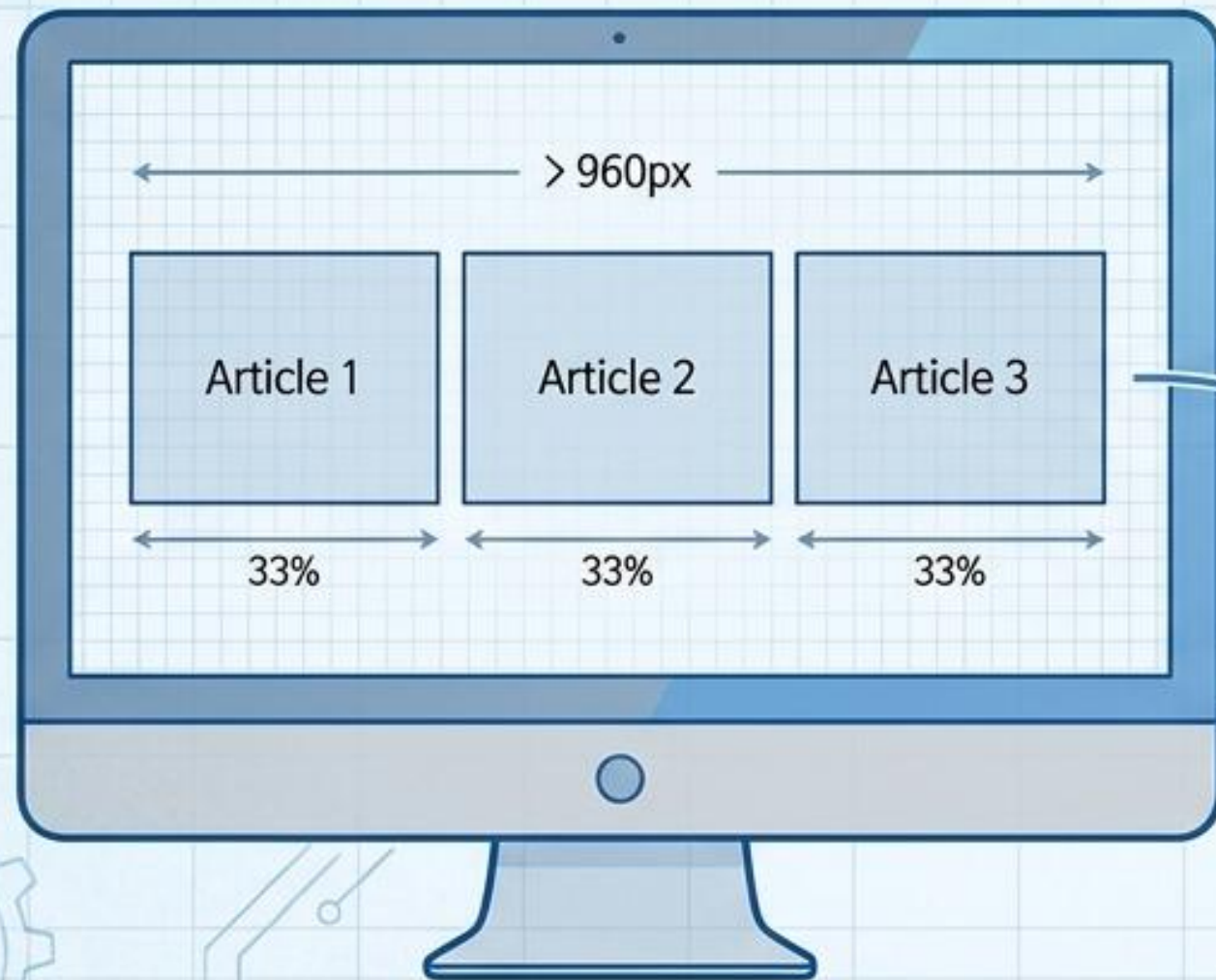
행이나 열 방향 중 하나의 축을 기준으로
내부 요소를 유연하게 균등 배치.

2차원 격자: display: grid

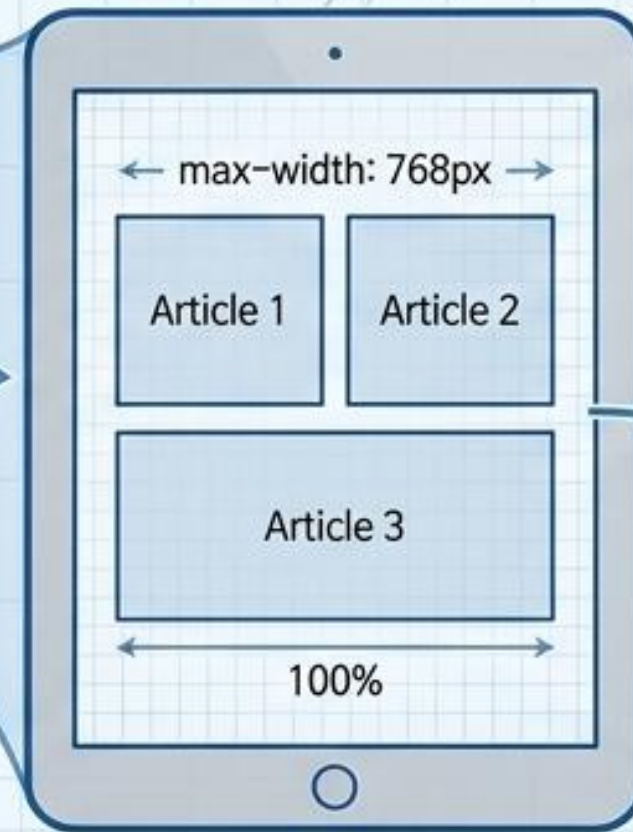


복잡한 2차원(행과 열 동시 제어)
구조의 레이아웃에 필수.

반응형 웹 블루프린트 (Responsive Web Design)



$> 960\text{px}$



$\text{max-width: } 768\text{px}$



$\text{max-width: } 480\text{px}$



디바이스 해상도에 반응하여 CSS 규칙을 갈아끼우는 미디어 쿼리를 통해 단일 소스로 다중 디바이스를 지원합니다.

UI 폴리싱: 상호작용(Interaction)의 디테일 완성하기

Form Elements



Focus Effect:
border & box-shadow

```
input:focus {  
  border-color: #1e90ff;  
  box-shadow: 0 0 10px rgba(30, 144, 255, 0.5);  
}
```

Button States

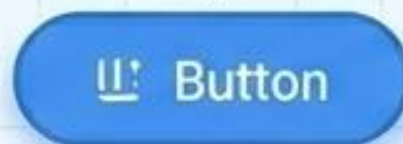
1단계 (Normal): 라운드 처리, 부드러운 그라데이션, 기본 그림자

```
box-shadow: 2px 2px 5px rgba(0,0,0,0.2);  
background: linear-gradient(135deg, #6a11cb, #2575fc);  
border-radius: 10px;
```



2단계 (Hover):

```
transform: translateY(-4px);  
box-shadow: 4px 4px 10px rgba(0,0,0,0.3);  
transition: all 0.3s ease;
```



3단계 (Active): 클릭 피드백

```
transform: translateY(1px);  
box-shadow: 1px 1px 3px rgba(0,0,0,0.2);
```

모서리의 라운드(border-radius), 미세한 그림자, 호버/포커스 전환(transition)은 평면적인 코드를 터치하고 싶은 '경험'으로 바꿉니다.

최종 합성: Request에서 Render까지의 거대한 여정



클릭 (Trigger)
브라우저가 Request 발송.



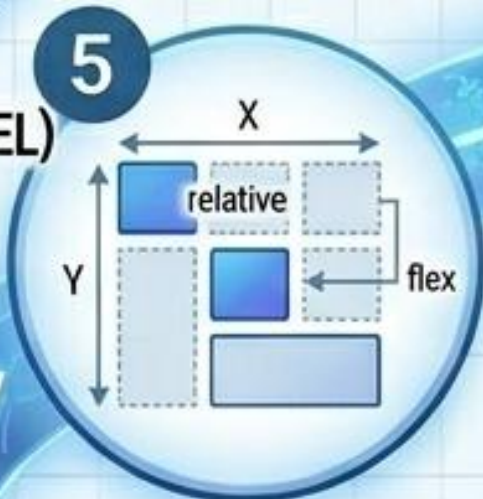
엔진 가동 (Tomcat)
서블릿 컨테이너가 `_jspService()` 실행. 파라미터 수신.



데이터 바인딩 (JSTL/EL)
DB 데이터가 동적 HTML 뼈대와 결합.



시멘틱 조립 (HTML)
완성된 시멘틱 HTML 문서로 응답(Response).



공간 배치 (Positioning)
CSS 엔진이 `relative`, `flex` 속성 등을 계산하여 좌표 설정.



최종 페인팅 (Painting)
Z-index 계산, 색상, 그림자 적용 후 화면에 픽셀 출력.

견고한 백엔드 로직 위에서,
정교한 프론트엔드 스타일링이 만날 때
완벽한 웹 애플리케이션이 탄생합니다.

모던 JSP 웹 아키텍처 설계도

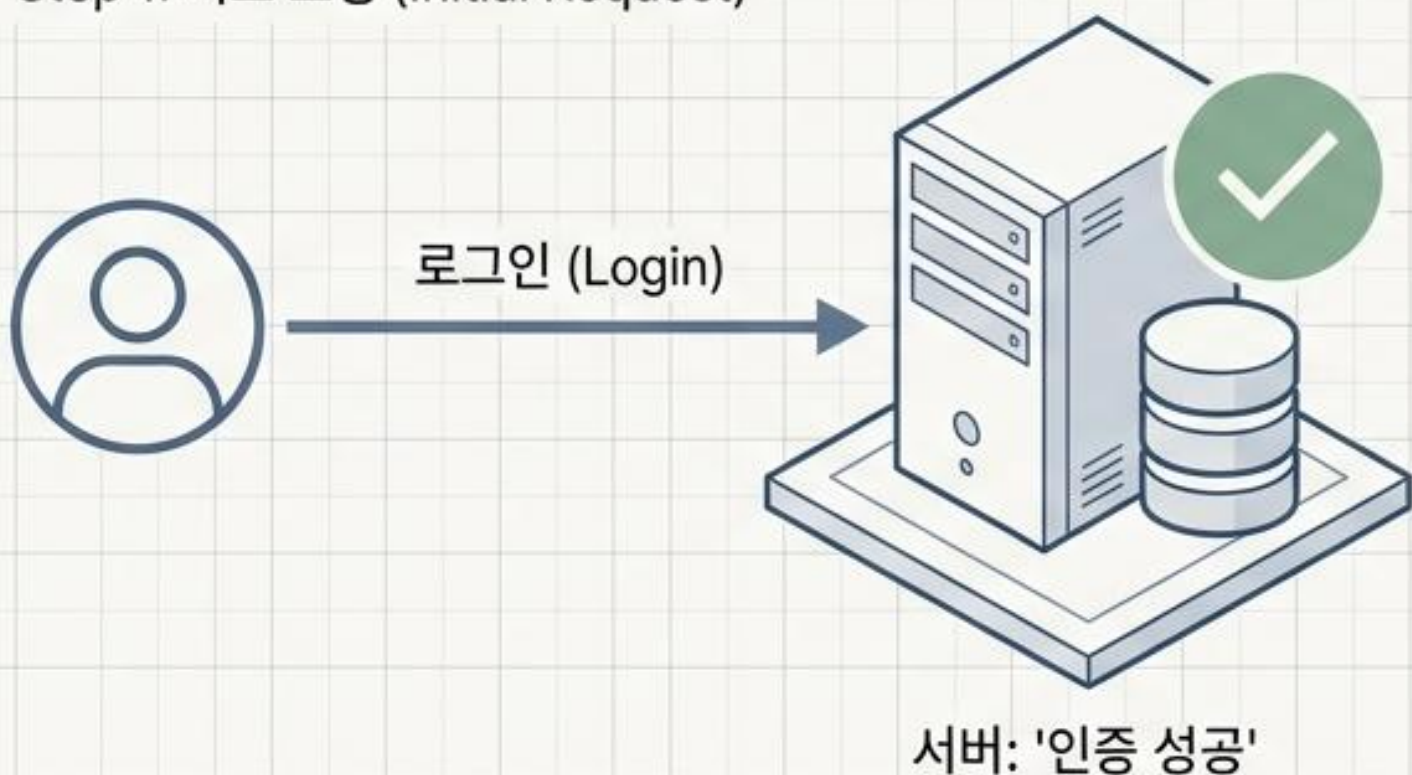
상태 관리, 데이터 연동, 그리고 MVC 패턴이 구성하는 웹의 기본 원리



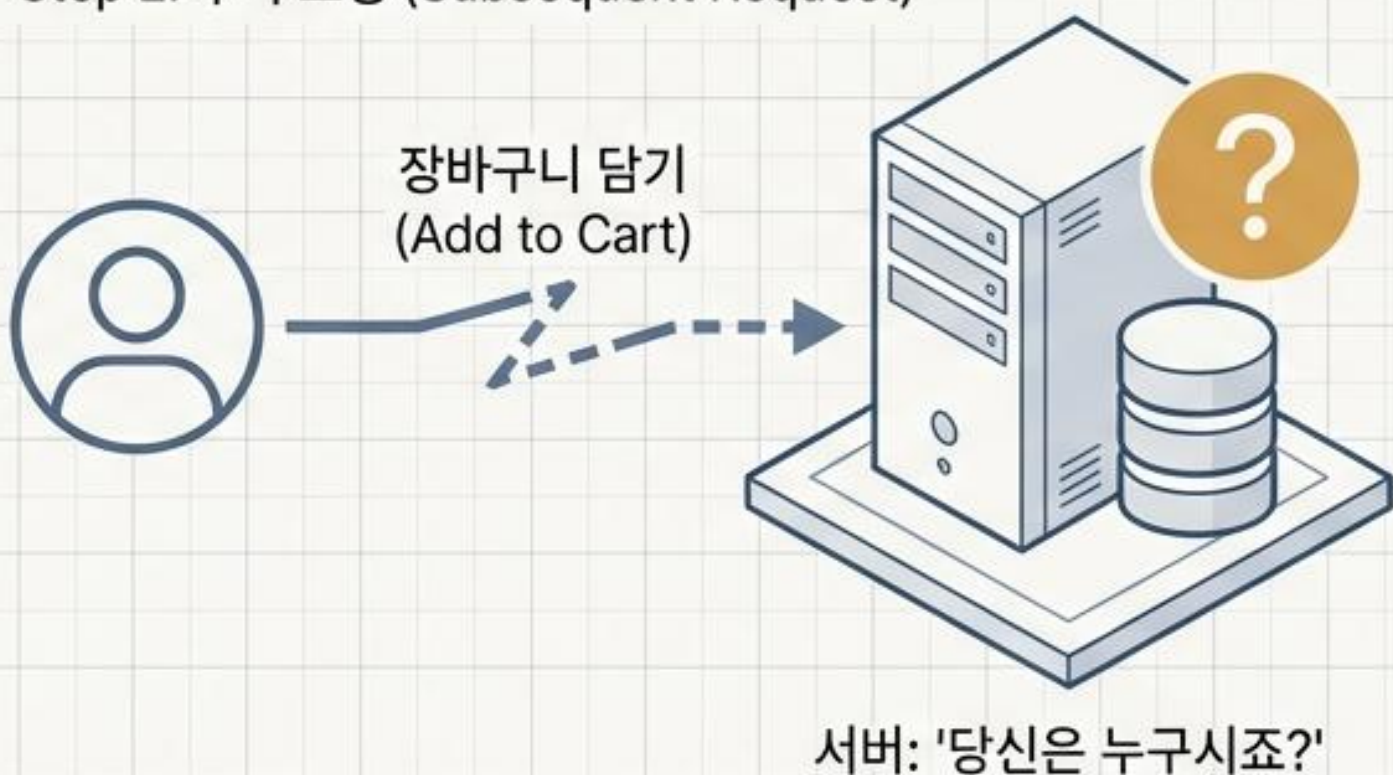
근본적인 문제: HTTP는 무상태(Stateless)입니다

서버는 이전 요청을 보낸 사용자를 기억하지 못합니다. 로그인 유지나 장바구니 기능을 구현하려면 서버와 클라이언트 간의 상태를 유지할 기억 장치가 필요합니다.

Step 1: 최초 요청 (Initial Request)



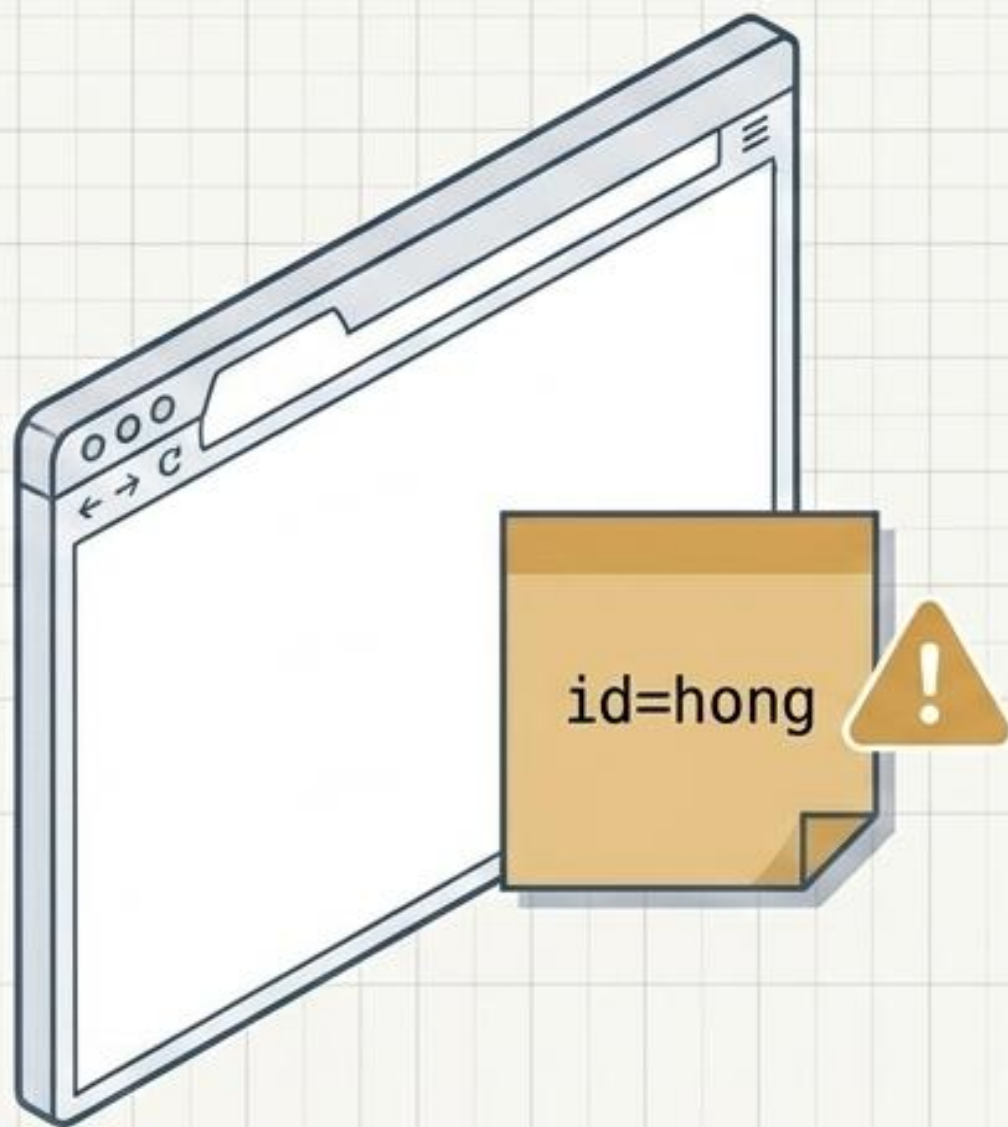
Step 2: 후속 요청 (Subsequent Request)



상태 유지를 위한 두 가지 핵심 기술: Cookie와 Session.

Cookie: 클라이언트 측 메모지

쿠키는 브라우저에 텍스트 형태로 저장되는 데이터입니다. 서버가 발급하지만, 보관과 전송은 전적으로 클라이언트(브라우저)가 담당합니다.



```
Cookie cookie = new Cookie("id", "hong");  
cookie.setMaxAge(60 * 60); // 1시간 유지  
response.addCookie(cookie);
```



저장 위치: 브라우저 (최대 약 4KB)



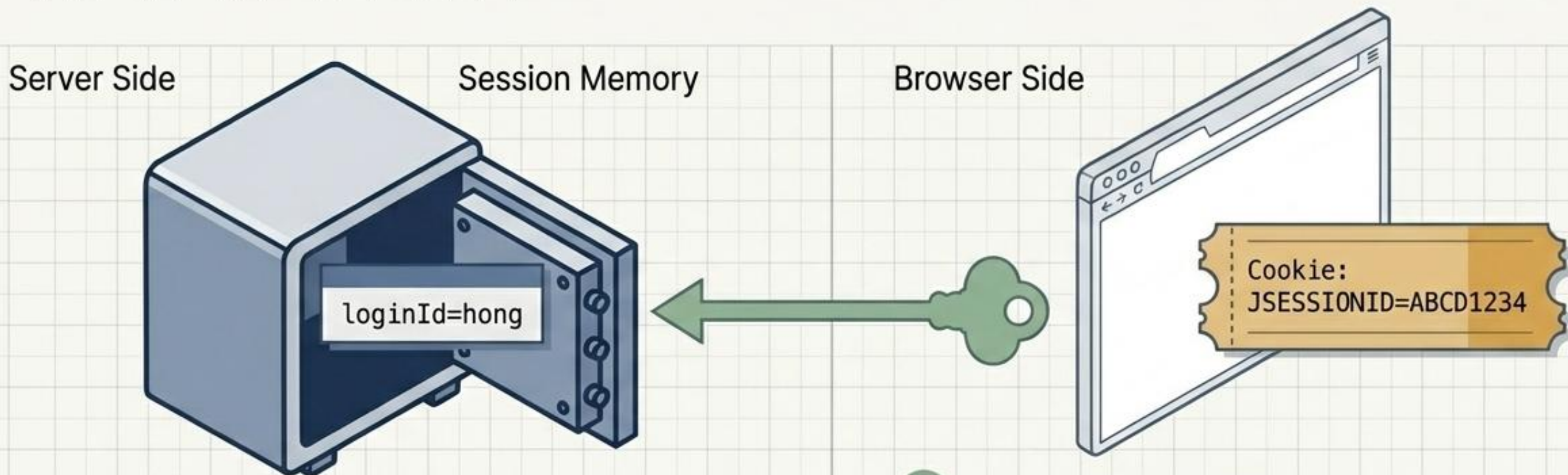
보안: 취약함 (사용자가 직접 수정 및 탈취 가능)



용도: 아이디 저장, 팝업 하루 안 보기 등
보안이 덜 중요한 정보.

Session: 서버 측 안전 금고

세션은 사용자 정보를 서버의 메모리에 안전하게 저장합니다. 브라우저에는 오직 금고 열쇠 역할을 하는 세션 ID만 부여됩니다.



```
session.setAttribute("loginId", "hong");  
// 로그아웃 시: session.invalidate();
```

- ✓ 저장 위치: 서버 메모리
- ✓ 보안: 상대적으로 매우 안전함
- ✓ 핵심: JSESSIONID라는 쿠키를 통해 브라우저와 세션을 연결합니다.

Cookie vs. Session 비교 분석

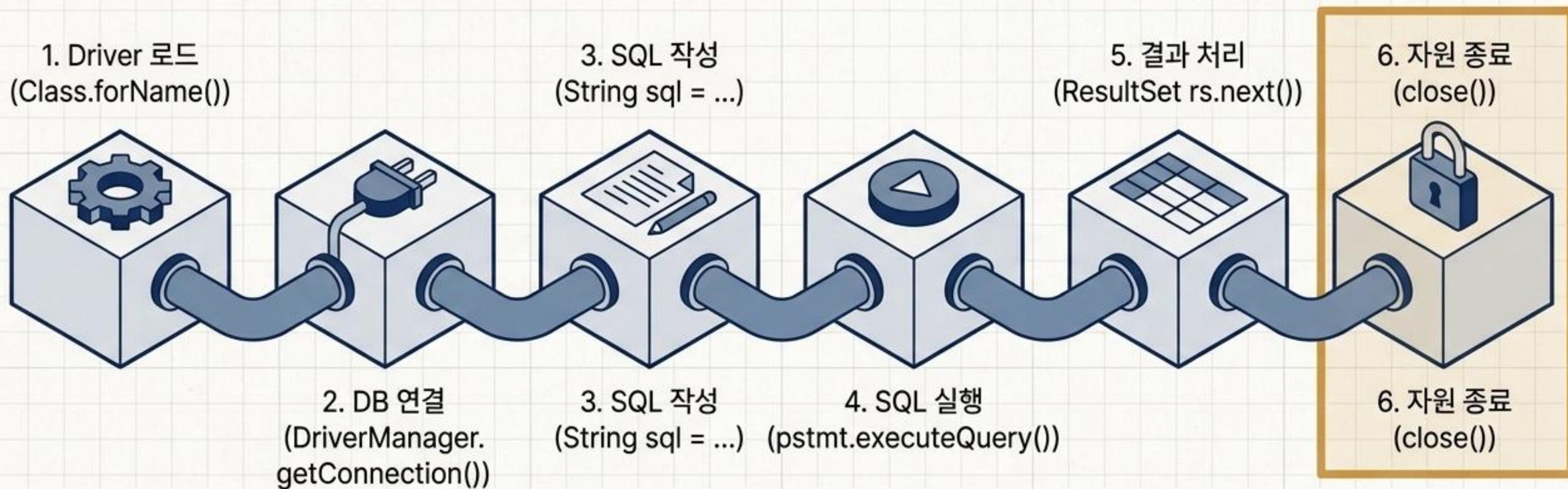
구분	저장 위치	보안성	저장 형식	속도	서버 부하
Cookie	브라우저	 취약 (탈취 위험)	텍스트 (약 4KB)	 빠름	 없음
Session	서버	 높음 (서버가 관리)	객체 (Object)	 상대적으로 느림	 메모리 차지

실무 웹 서버의 표준 구조

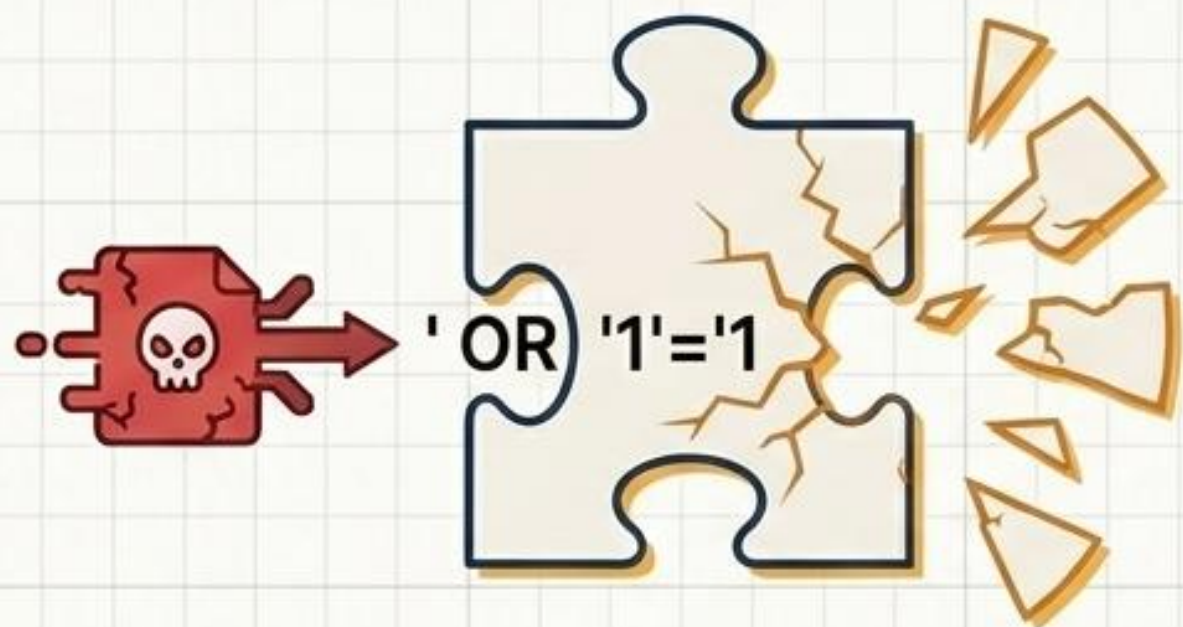
Cookie와 Session은 경쟁 관계가 아닙니다. 서버는 Session으로 데이터를 보호하고, 그 Session을 식별하기 위한 열쇠(Session ID)를 Cookie로 브라우저에 전달합니다. (예: JSESSIONID)

데이터 영구 저장: JDBC 6단계 파이프라인

Java와 DBMS를 연결하는 표준 API인 JDBC(Java Database Connectivity)의 생명주기입니다.



보안 위협: Statement vs PreparedStatement



위험: Statement

단순 문자열 결합 방식으로 SQL 작성.
SQL Injection 공격에 취약.

```
select * from member where id=' ' + id + ' '
```

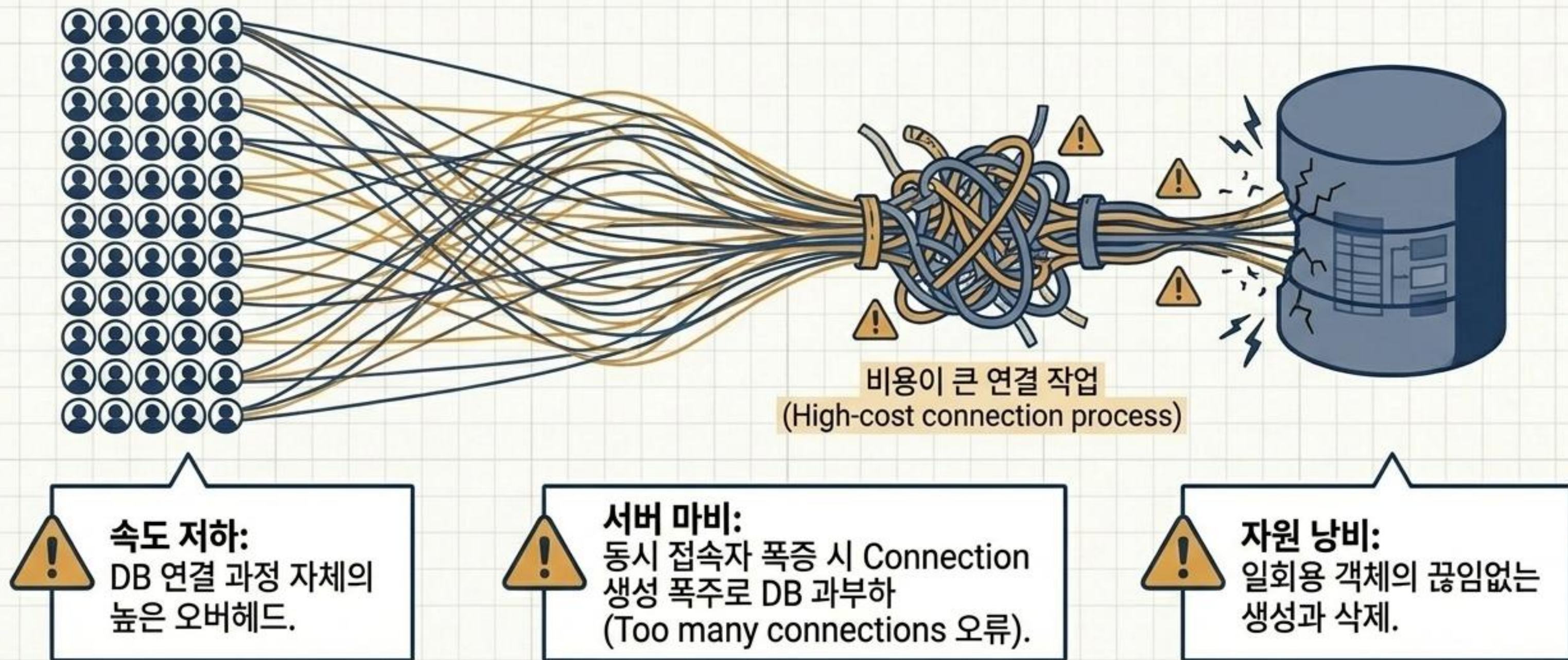
안전: PreparedStatement

실무 표준. 파라미터를 ?로 바인딩하여
쿼리 구조를 미리 컴파일.

```
select * from member where id=?  
pstmt.setString(1, id); -> 악의적 코드가 무력화됨.
```

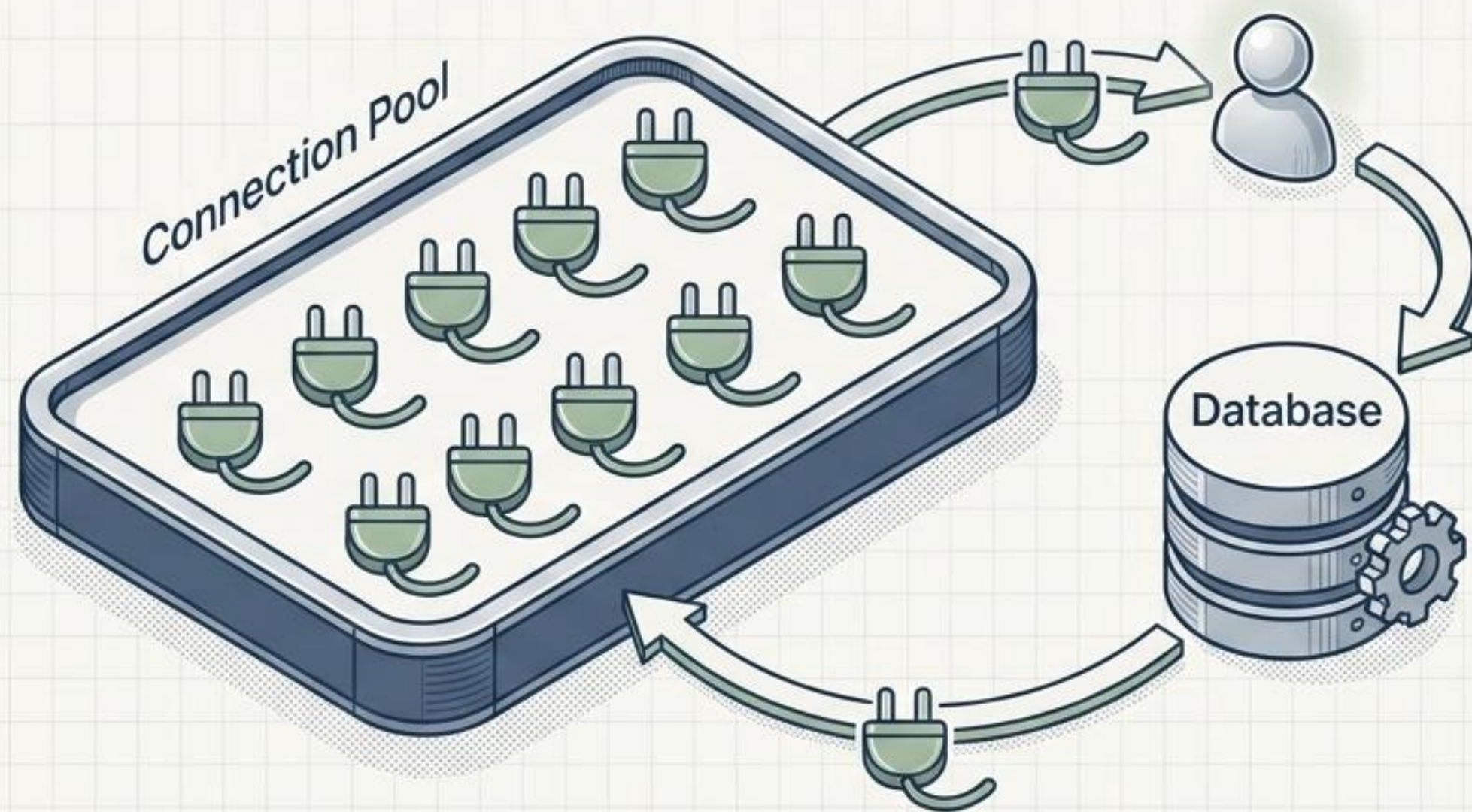
기존 JDBC의 병목 현상 (Bottleneck)

매 요청마다 DB 연결(Connection)을 새로 생성하고 파기하는 것은 시스템에서 가장 무겁고 느린 작업입니다.



해결책: 커넥션 풀 (Connection Pool)

요청마다 생성하지 않습니다. 서버 시작 시 연결을 미리 여러 개 생성해두고(Pool), 요청이 올 때마다 대여해 주고 반납받는 재사용 구조입니다.



핵심 이점 (Key Benefits)

- ⬆ **성능 향상**: 연결 생성 **비용 제로** (즉시 응답).
- 🛡 **안정성**: 최대 Connection 수를 제한하여 DB 과부하 방지.
- ⚙ **실무 표준**: 현재 Spring Boot의 기본 풀은 매우 가볍고 **빠른 HikariCP**입니다.

DataSource와 반납의 의미



JNDI DataSource

```
// 1. DataSource (공장) 획득  
DataSource ds = (DataSource)ctx.lookup("jdbc/mydb");
```

```
// 2. Pool에서 Connection 대여  
Connection conn = ds.getConnection();
```

```
// ... SQL 실행 ...
```

```
// 3. 자원 반납  
conn.close();
```

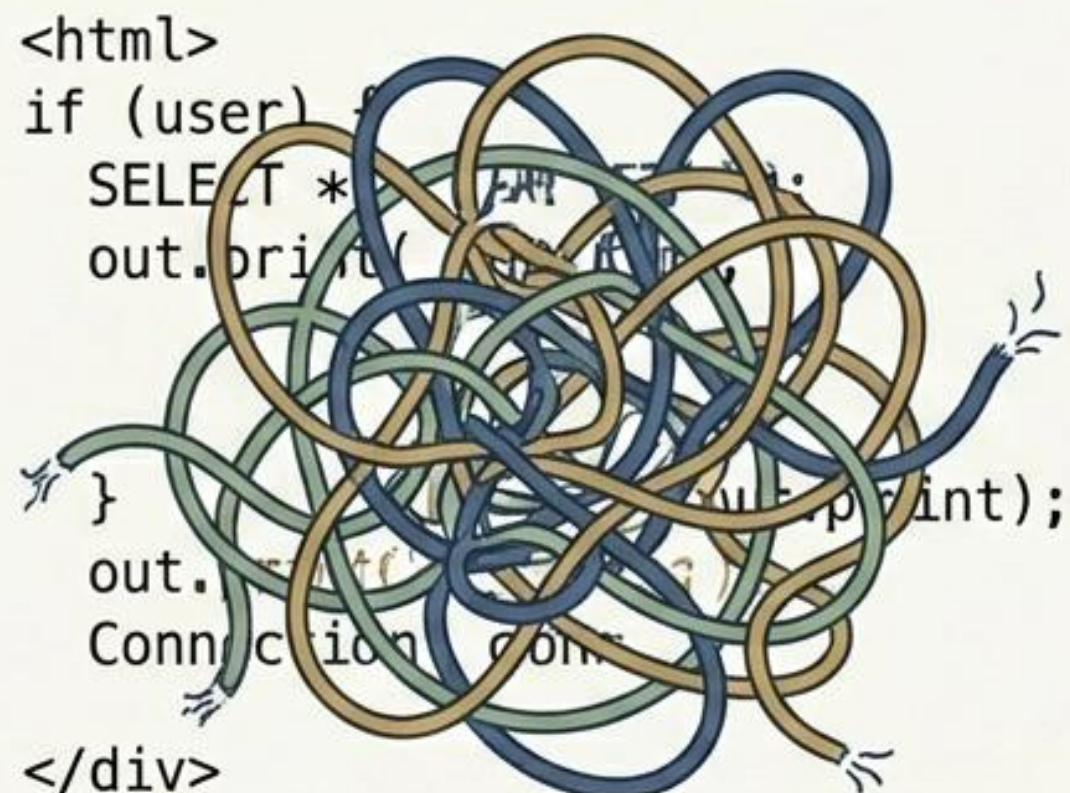
Connection Pool 환경에서 close()는
실제 연결 종료가 아닙니다.
Pool로의 반납을 의미합니다.

아키텍처의 부재: Model 1의 재앙 (Spaghetti Code)

초기 웹 개발 방식에서는 하나의 JSP 파일 안에 화면 구성(HTML), 비즈니스 로직(Java), DB 접근(SQL)이 모두 섞여 있었습니다.

JSP (Model 1)

```
<html>
if (user) {
  SELECT * FROM ...;
  out.print(
}
out.print(
Connection con
</div>
```



유지보수 불가

코드 복잡도가 극도로 높아 수정이 어렵습니다.



역할 분리 실패

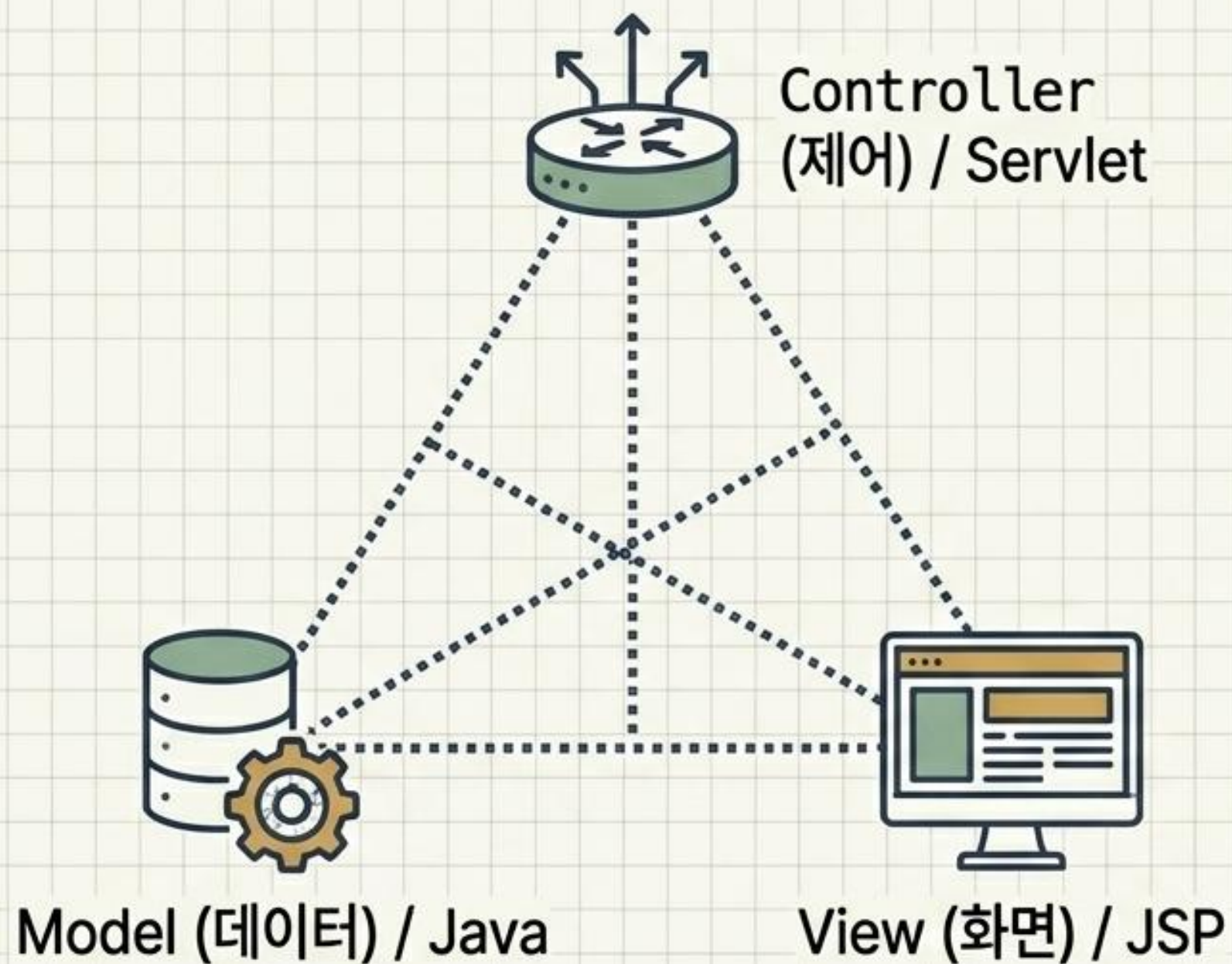
디자이너와 백엔드 개발자의 협업이 불가능합니다.



재사용성 제로

코드가 화면에 종속되어 재사용할 수 없습니다.

관심사의 분리: MVC 패턴 (Model 2)



Model (데이터/로직): 데이터베이스 접근 및 비즈니스 로직을 처리합니다. (DTO, DAO, Service)

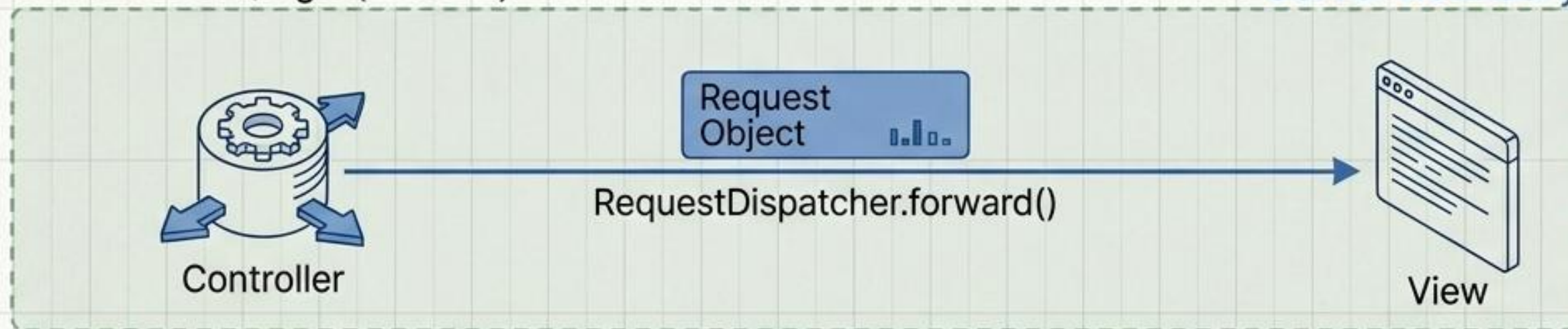
View (화면): 처리된 결과를 사용자에게 보여주는 화면 출력만 담당합니다. (JSP 안에는 DB 코드가 없습니다)

Controller (제어): 사용자의 요청을 분석하고, Model을 호출하며, 결과를 View로 전달하는 두뇌 역할을 합니다. (Servlet)

Controller의 라우팅 기술: Forward vs Redirect

URL: /login (변경 안됨)

Server Boundary



Forward (RequestDispatcher.forward())

서버 내부 이동. 브라우저 URL이 변경되지 않으며, request 객체의 데이터가 유지됩니다.
MVC에서 데이터를 View로 넘길 때 핵심적으로 사용.



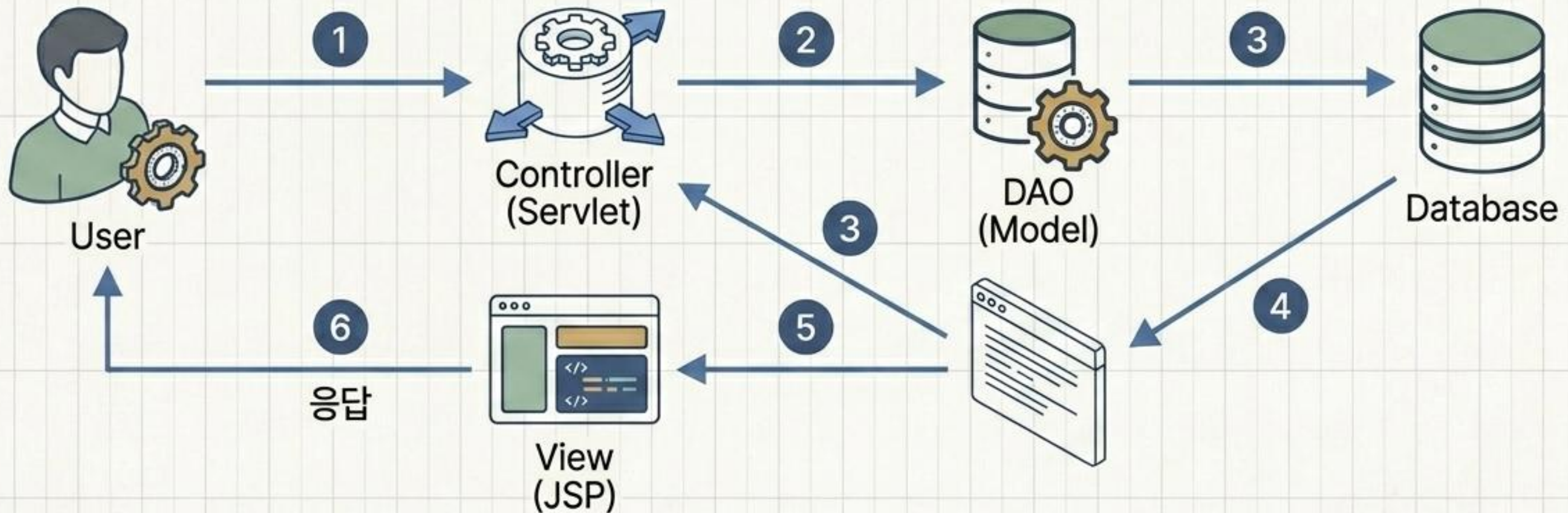
Browser



Redirect (response.sendRedirect())

브라우저 재요청. 브라우저 URL이 변경되며, 기존 request 데이터는 사라집니다.

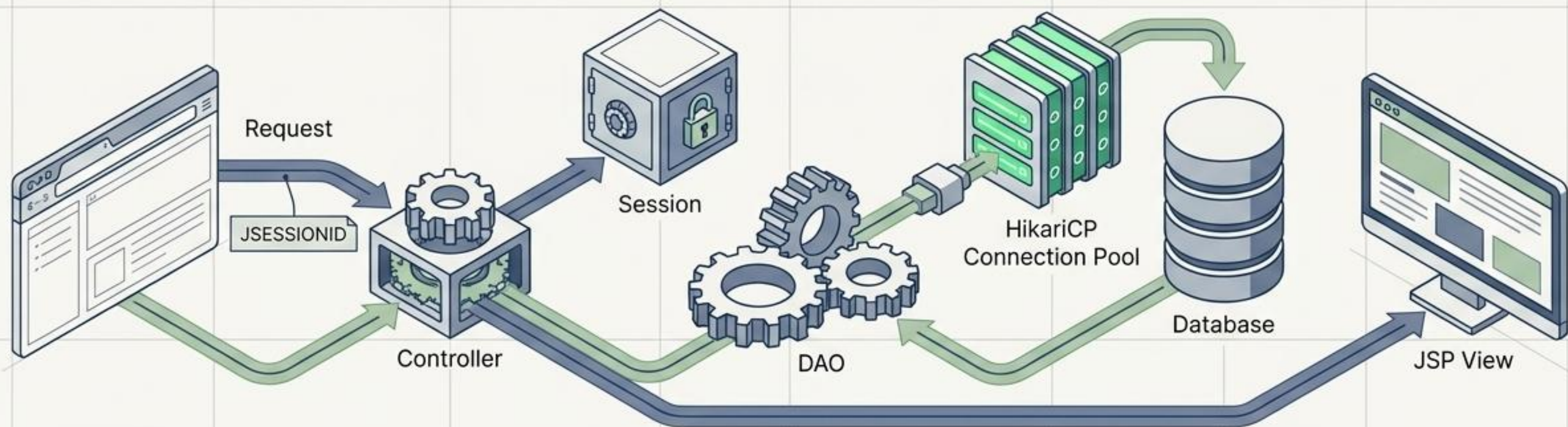
모던 MVC 요청 처리 흐름



1. **요청**: 브라우저가 Controller(Servlet)에 요청 전달.
2. **분석 및 호출**: Controller가 요청을 분석하고 Model(DAO) 호출.
3. **DB 연동**: DAO가 데이터베이스에서 정보 조회.

4. **결과 반환**: DB 결과가 DTO 객체로 Controller에 반환됨.
5. **저장 및 라우팅**: Controller가 데이터를 request에 저장 후 View(JSP)로 **Forward**.
6. **응답**: JSP가 데이터를 **HTML**로 렌더링하여 최종 응답.

Master Architecture: 모든 톱니바퀴의 결합



우리가 배운 모든 기술은 하나의 시스템 안에서 완벽하게 맞물려 작동합니다.

1. Session이 사용자를 인증하고,
2. Controller가 흐름을 제어하며,
3. Connection Pool이 성능 하락 없이 DB 데이터를 가져와,
4. View가 화면을 그립니다.

Spring Boot로의 진화: 이 Servlet/JSP 기반의 근본적인 구조는 오늘날 Spring MVC, JPA, MyBatis 내부에서도 동일한 원리로 작동하는 현대 웹 아키텍처의 절대적인 기반입니다.